

Министерство науки и высшего образования Российской Федерации
Муромский институт (филиал)
федерального государственного бюджетного образовательного учреждения высшего образования
«Владимирский государственный университет
имени Александра Григорьевича и Николая Григорьевича Столетовых»
(МИ ВлГУ)

Факультет _____ ИТР _____
Кафедра _____ ИС _____

КУРСОВАЯ РАБОТА

По дисциплине: Прикладная разработка на Java

Тема: Разработка приложения для визуализации системы умного дома

Оценка работы

Руководитель
_____ Метёлкин А.С.
(фамилия, инициалы)

Члены комиссии

_____ (подпись) _____ (дата)
Студент _____ ИС-123
(группа)
_____ Зуев Е. И.
(фамилия, инициалы)

_____ (фамилия, инициалы) _____ (подпись)

_____ (фамилия, инициалы) _____ (подпись)

_____ (подпись) _____ (дата)

В курсовой работе отражено создание программного модуля "3D визуализация" для конфигуратора системы "Умный дом", реализованного на языке Java с использованием библиотек LWJGL и JavaFX. В ходе работы были исследованы принципы интерактивной трехмерной графики и способы интеграции 2D-редактора с 3D-рендерером. Разработана структура данных для хранения информации о здании, включающая стены, комнаты, предметы интерьера и устройства умного дома, и реализованы основные операции редактирования: создание стен, формирование помещений, размещение мебели и датчиков.

Разработанное приложение включает 2D-редактор плана здания с инструментами рисования и привязкой к сетке, 3D-рендерер на базе OpenGL для просмотра созданной модели в реальном времени, а также систему процедурной генерации текстур (кирпичная кладка, дерево, дверь) для повышения реалистичности сцены. Для демонстрации возможностей системы реализованы три режима симуляции работы устройств: случайный, "умный дом" и ручной, обеспечивающие визуальную обратную связь (изменение состояния ламп, срабатывание датчиков движения, освещенности и температуры).

Модуль 3D-визуализации может быть полезен проектировщикам систем автоматизации зданий, дизайнерам интерьера и конечным пользователям для наглядного представления планировки помещений, расположения устройств и сценариев их работы. Приложение также служит примером реализации полнофункциональной системы управления трехмерными данными с элементами симуляции в среде Java.

Содержание

Введение.....	6
1 Анализ технического задания	8
1.1 Анализ существующих аналогов и их функций	8
1.2 Выбор стека технологий.....	12
2 Проектирование программы	13
2.1 Архитектура приложения.....	13
3 Разработка программы.....	18
3.1 Разработка 2D-редактора плана здания	18
3.2 Реализация 3D-рендерера на базе LWJGL и OpenGL	22
3.3 Разработка и интеграция симуляторов устройств умного дома	28
3.4 Разработка базы данных	35
4 Тестирование	41
4.1 Модульное тестирование (Unit-тесты)	41
4. 2 Ручное функциональное тестирование	42
Заключение	44
Список использованных источников	46
Приложение А	47

					<i>МИВУ.09.03.02-06.000 ПЗ</i>			
<i>Изм.</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Подпись</i>	<i>Дата</i>	<i>Разработка приложения для визуализации системы умного дома</i>	<i>Лит.</i>	<i>Лист</i>	<i>Листов</i>
<i>Разраб.</i>		<i>Зуев Е. И.</i>						
<i>Провер.</i>		<i>Метелкин А.С.</i>					<i>5</i>	<i>58</i>
<i>Н. Контр.</i>						<i>МИ ВлГУ</i>		
<i>Утверд.</i>						<i>ИС-123</i>		

Введение

В современном мире технологии «Умный дом» становятся неотъемлемой частью повседневной жизни, обеспечивая комфорт, энергоэффективность и безопасность. С развитием интернета вещей и систем автоматизации интерес к созданию эффективных и наглядных инструментов для проектирования и мониторинга таких систем продолжает расти. Современные подходы, такие как использование трехмерной визуализации с интегрированными симуляторами устройств, позволяют разрабатывать решения, которые могут быть легко адаптированы для различных сценариев использования – от частных домов до офисных центров [1-4].

Тема курсовой работы – разработка приложения для визуализации системы умного дома с использованием языка Java, библиотек LWJGL и JavaFX. Объектом исследования является процесс проектирования и реализации интерактивной системы визуализации, включающий создание 2D-редактора плана здания, генерацию трехмерной сцены с процедурными текстурами и интеграцию симуляторов устройств. Предметом исследования является разработанный модуль визуализации, обеспечивающий наглядное представление планировки помещений, расположения устройств и их текущего состояния.

Цель курсовой работы – изучение и разработка полнофункционального модуля 3D-визуализации для конфигуратора системы «Умный дом», реализованного с использованием современных технологий: Java для серверной логики, LWJGL/OpenGL для трехмерной графики, JavaFX для пользовательского интерфейса и SQLite для хранения данных о проектах. В рамках работы будет создан программный продукт, который предоставляет комплексный инструментарий для проектирования планировки помещений и визуализации работы устройств умного дома.

Для достижения поставленной цели необходимо решить следующие задачи [1-4]:

					<i>МИВУ.09.03.02-06.000 ПЗ</i>	Лист
Изм.	Лист	№ докум.	Подп.	Дата		6

1) разработка структуры данных для хранения информации о здании (стены, комнаты, предметы интерьера) и устройствах умного дома (лампы, датчики движения, температуры, освещенности, дыма);

2) создание архитектуры приложения с разделением на клиентскую (JavaFX UI) и рендер-часть (LWJGL/OpenGL);

3) разработка 2D-редактора плана здания с возможностью рисования стен, создания комнат, размещения мебели и устройств;

4) реализация 3D-рендерера для отображения сгенерированной сцены с возможностью управления камерой (орбитальный режим и режим «от первого лица»);

5) разработка системы процедурной генерации текстур для стен (кирпичная кладка), пола (дерево) и дверей;

6) реализация библиотеки предметов интерьера и устройств с поддержкой различных типов крепления (пол, стена, потолок);

7) создание трёх режимов симуляции работы устройств: случайный, «умный дом» (автоматическое управление) и ручной (управление кликом мыши);

8) реализация системы сохранения и загрузки проектов в JSON-формат и базу данных;

9) тестирование и отладка приложения, проверка работоспособности всех функций и оптимизация производительности рендерера.

Результатом работы станет полнофункциональный модуль 3D-визуализации для конфигуратора системы «Умный дом», позволяющий эффективно проектировать планировку помещений и визуализировать работу устройств, а также база данных, обеспечивающая надежное хранение и обработку информации о проектах.

					<i>МИВУ.09.03.02-06.000 ПЗ</i>	<i>Лист</i>
<i>Изм.</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Подп.</i>	<i>Дата</i>		<i>7</i>

1 Анализ технического задания

Трёхмерная визуализация систем «Умный дом» – это процесс, при котором информация о планировке здания, расположении устройств и их текущем состоянии преобразуется в наглядную трёхмерную сцену для эффективного восприятия и анализа пользователем. Этот процесс включает такие операции, как моделирование стен и комнат по 2D-контуру, наложение текстур на поверхности, размещение трёхмерных моделей мебели и устройств, а также динамическое обновление визуальных характеристик объектов в зависимости от их состояния (включена/выключена лампа, сработал датчик движения). Примером визуализации может служить создание интерактивной 3D-сцены квартиры с возможностью «прогулки» по комнатам и наблюдения за работой автоматизации.

1.1 Анализ существующих аналогов и их функций

Перед разработкой модуля визуализации системы «Умный дом» был проведён сравнительный анализ существующих решений в данной области [2]. В качестве аналогов рассматривались три наиболее популярных инструмента.

Первым рассматривался Sweet Home 3D – программа для проектирования интерьеров, которая позволяет создавать 2D-план с автоматической генерацией 3D-вида. Данное решение включает библиотеку мебели и объектов интерьера, поддерживает управление камерой (вращение и приближение), а также импорт дополнительных 3D-моделей в формате OBJ и сохранение проектов в файл.

Вторым аналогом выступил Home Assistant с дополнением Floorplan – система умного дома, предоставляющая 2D-схему помещений с привязкой устройств. Ключевыми возможностями являются отображение состояния датчиков с цветовой индикацией, интеграция с реальными устройствами умного дома и поддержка сенсорного управления.

					МИВУ.09.03.02-06.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		8

Третьим аналогом стал Google SketchUp (Trimble) – профессиональное средство для 3D-моделирования зданий. Продукт отличается большой библиотекой готовых моделей (3D Warehouse), возможностью экспорта в различные форматы, а также высокой детализацией и реалистичностью сцен.

Недостатки существующих аналогов.

Sweet Home 3D: не предназначен для интеграции с симуляторами устройств, отсутствует динамическое обновление состояния датчиков в реальном времени, нет поддержки сценариев автоматизации.

Home Assistant + Floorplan: требует ручного конфигурирования координат устройств в коде, не имеет встроенного 2D-редактора, 3D-визуализация отсутствует.

Google SketchUp: слишком сложен для рядового пользователя, требует профессиональных навыков 3D-моделирования, не имеет встроенных средств для работы с системой «Умный дом».

Вывод: существующие решения либо не обеспечивают достаточной наглядности (только 2D), либо сложны в освоении, либо не поддерживают динамическую симуляцию работы устройств. Это обуславливает необходимость создания собственного легковесного модуля 3D-визуализации, интегрированного с симуляторами умного дома.

Функциональные требования на основе анализа аналогов

На основе анализа существующих систем были определены ключевые функции, которые должны быть реализованы в разрабатываемом модуле визуализации «Умный дом»:

- создание и редактирование 2D-плана здания (стены, комнаты);
- автоматическая генерация 3D-моделей стен по 2D-контур;
- библиотека предметов интерьера и устройств (мебель, лампы, датчики);
- процедурная генерация текстур (кирпич, дерево, дверь);
- размещение устройств с учётом типа крепления (пол, стена, потолок);
- симуляция работы устройств умного дома;

					МИВУ.09.03.02-06.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		9

- визуальная обратная связь от устройств (изменение цвета ламп);
- управление камерой в 3D-режиме;
- сохранение и загрузка проектов.

Управление данными о зданиях и устройствах является важной областью информационных систем, которая описывается реляционными моделями и алгоритмическими подходами. Один из классических методов, используемых в системах 3D-визуализации, - представление здания в виде графа, где стены являются рёбрами, а комнаты – гранями.

В контексте разработки программного обеспечения для 3D-визуализации умного дома, приложения с использованием OpenGL и JavaFX представляют собой эффективный инструмент, предоставляющий комплексный набор функций для выполнения основных операций, таких как создание планировки, управление камерой, рендеринг текстур и симуляция устройств. Эти системы основываются на принципах клиент-серверной архитектуры, где данные хранятся в структурированном виде и обрабатываются с помощью бизнес-логики приложения.

Процесс визуализации в рамках данного приложения включает два основных аспекта.

Редактирование 2D-плана – выполнение операций, таких как создание стен (кликами мыши), формирование замкнутых комнат, размещение предметов из библиотеки. Пример интерфейса 2D-редактора показан на рисунке 1.

3D-просмотр и симуляция – организация интерактивного просмотра сгенерированной сцены с возможностью управления камерой, визуализация состояния устройств (лампы горят/не горят, датчики активны/неактивны). Пример 3D-вида представлен на рисунке 2.

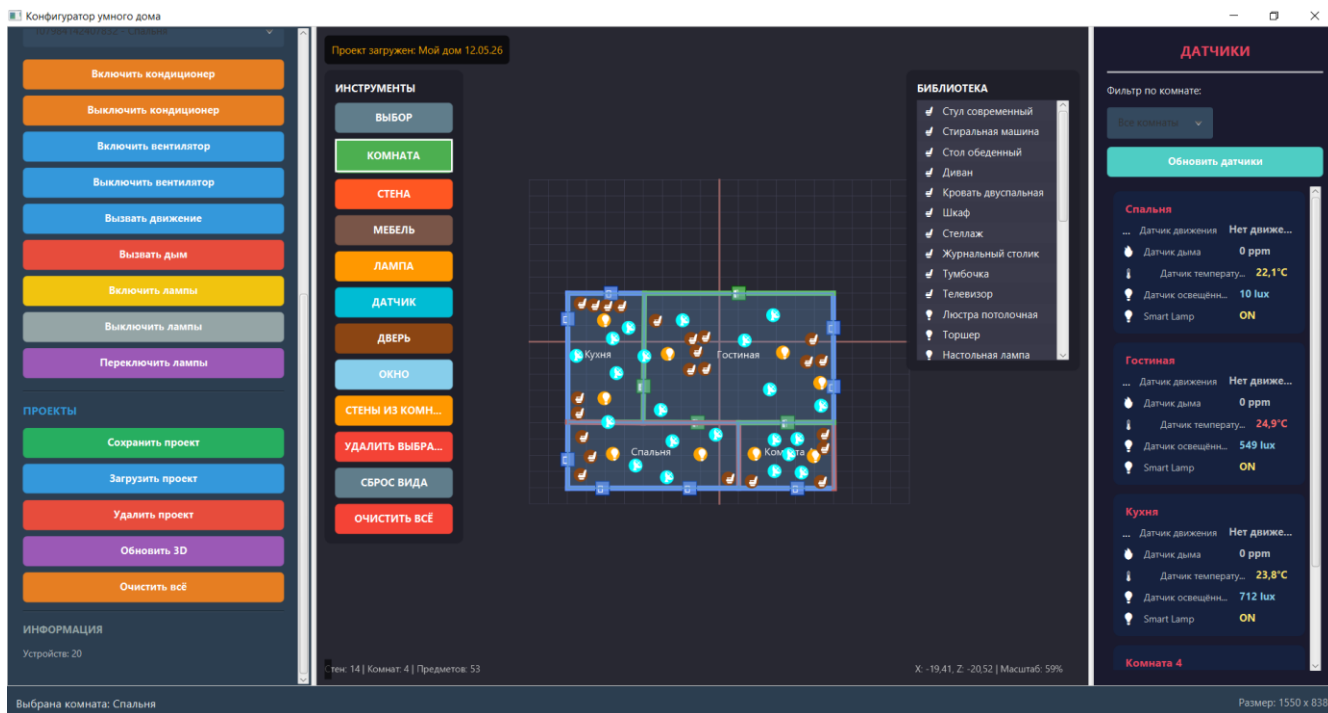


Рисунок 1 – Пример интерфейса 2D-редактора



Рисунок 2 – Пример 3D-вида

Алгоритмы обработки данных о зданиях и устройствах, реализованные в приложении, опираются на объектно-ориентированный подход, где каждая сущность представлена в виде класса на языке Java. Основная цель приложения – обеспечить выполнение операций по визуализации и симуляции, минимизируя

временные затраты пользователя и обеспечивая высокую производительность рендерера.

1.2 Выбор стека технологий

Для реализации модуля 3D-визуализации системы «Умный дом» был выбран следующий стек технологий.

Язык программирования Java обеспечивает кроссплатформенность, высокую производительность благодаря JIT-компиляции, встроенные средства многопоточности и обширную экосистему библиотек.

JavaFX используется для создания пользовательского интерфейса и 2D-редактора плана здания. Класс Canvas позволяет реализовать рисование стен, создание комнат, размещение предметов с привязкой к точкам, масштабированием и панорамированием. JavaFX также обеспечивает реактивную синхронизацию интерфейса с моделью данных.

LWJGL (Lightweight Java Game Library) – основная библиотека для 3D-рендеринга. Она предоставляет низкоуровневый доступ к OpenGL (шейдеры, VAO/VBO-буферы, текстуры) и GLFW (создание отдельного окна в своём потоке). Выбор LWJGL обусловлен необходимостью полного контроля над графическим конвейером и высокой производительностью.

SQLite – встраиваемая реляционная база данных для хранения проектов, стен, комнат, предметов мебели, устройств и истории событий. Не требует отдельного сервера, данные хранятся в одном файле, что упрощает развёртывание и перенос проектов.

JUnit 4 используется для модульного тестирования геометрических алгоритмов, классов сущностей и логики симуляторов.

Jackson (ObjectMapper) – библиотека для сериализации проектов в JSON-формат при экспорте и импорте.

					<i>МИВУ.09.03.02-06.000 ПЗ</i>	Лист
Изм.	Лист	№ докум.	Подп.	Дата		12

Альтернативы (Python + Pygame, C# + Unity, Swing) были отклонены из-за недостаточной производительности, сложности интеграции с OpenGL или отсутствия необходимой гибкости для реализации кастомного 2D-редактора и многопоточного рендерера.

2 Проектирование программы

2.1 Архитектура приложения

Архитектура программного модуля 3D-визуализации для конфигуратора системы «Умный дом» построена по классической многоуровневой модели, разделяющей систему на три логических уровня: представления, логики и данных. Такой подход обеспечивает модульность, возможность расширения и удобство сопровождения кода.

На рисунке 3 представлена общая архитектура приложения.

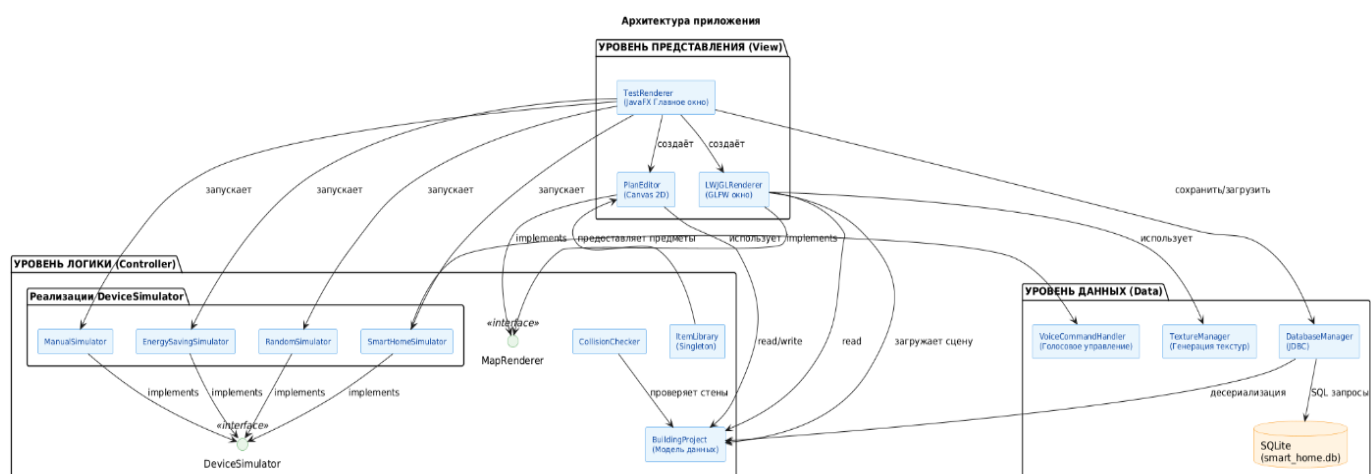


Рисунок 3 – Архитектура приложения

Взаимодействие между компонентами приложения организовано через чётко определённые интерфейсы.

На рисунке 8 в приложении А показана схема взаимодействия интерфейсов.

Описание взаимодействия [2-3]:

1) интерфейс MapRenderer – определяет контракт для всех рендереров:

- render(Project project, Pane container) – отрисовка в переданную панель;
- clear() – очистка сцены;
- getName() – возвращает название режима;
- handleClick(double x, double y) – обработка клика.

Этот интерфейс реализуют классы PlanEditor (2D-рендерер), LWJGLRenderer (3D-рендерер) и SvgExporter (экспорт в SVG).

2) интерфейс DeviceSimulator – определяет контракт для симуляторов устройств:

- startSimulation(Project project) – запуск симуляции;
- stopSimulation() – остановка;
- getName() – название режима.

Реализации: RandomSimulator, SmartHomeSimulator, ManualSimulator, EnergySavingSimulator.

3) взаимодействие TestRenderer (главный контроллер):

- содержит ссылки на текущий рендерер и текущий симулятор;
- при нажатии кнопки «2D План» вызывает switchToPlanEditor() – создаёт PlanEditor, устанавливает его как currentRenderer и вызывает render();
- при нажатии кнопки «3D Вид» вызывает switchTo3DView() – создаёт LWJGLRenderer, передаёт ему currentBuildingProject и текущий симулятор;
- при сохранении проекта вызывает DatabaseManager.saveBuildingData() и DatabaseManager.saveProject();
- при выборе симулятора вызывает switchSimulator(), который останавливает текущий симулятор, создаёт новый и передаёт его в LWJGLRenderer;
- на правой панели отображает актуальные показания всех датчиков в реальном времени.

4) взаимодействие PlanEditor ↔ BuildingProject:

- PlanEditor хранит ссылку на BuildingProject;

- при рисовании стены добавляет WallSegment в buildingProject.getWalls();
- при создании комнаты добавляет RoomArea в buildingProject.getRooms();
- после каждого изменения вызывает draw() для обновления 2D-вида.

Взаимодействие LWJGLRenderer ↔ BuildingProject:

5) метод loadBuildingProject(BuildingProject project) вызывается из основного потока;

- внутри формируется задача Runnable, которая помещается в BlockingQueue <Runnable> glTasks;

- поток рендеринга забирает задачу, захватывает sceneLock и перестраивает OpenGL-буферы (VBO, VAO, EBO);

- при отрисовке кадра sceneLock защищает доступ к спискам meshes и materials.

6) взаимодействие TextureManager с рендерами:

- реализован как синглтон (паттерн Singleton);

- PlanEditor не использует текстуры (только плоские цвета);

- LWJGLRenderer при создании меша вызывает generateWoodTexture (), generateDoorTexture () или TextureManager.generateBrickTexture();

- текстуры кэшируются в textureCache, повторные вызовы возвращают существующий ID.

7) взаимодействие с ItemLibrary:

- синглтон, загружает все предметы при запуске;

- PlanEditor использует её для отображения списка в ListView и получения размеров предмета при проверке коллизий;

- при размещении предмета создаётся FurnitureItem с libraryId из выбранного элемента.

8) взаимодействие с DatabaseManager:

- синглтон, использует ObjectMapper (Jackson) для JSON-сериализации;

- при сохранении: BuildingProject → JSON → SQLite (таблица BUILDING_PROJECTS);

					МИВУ.09.03.02-06.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		15

- дополнительно сохраняет нормализованные данные в таблицы: WALL_SEGMENTS, ROOM_AREAS, ROOM_AREA_VERTICES, FURNITURE_ITEMS;
- при загрузке: SQLite → JSON → BuildingProject и загрузка связанных таблиц;
- поддерживает каскадное удаление проектов.

9) взаимодействие с голосовым управлением (SmartHomeSimulator + VoiceCommandHandler):

- при активации симулятора «Умный дом» запускается VoiceCommandHandler;
- распознаёт фразы, начинающиеся с «Умный дом» (например, «Умный дом включи свет в гостиной»);
- поддерживает команды: включение/выключение света, регулировка яркости, управление кондиционером и вентилятором, переключение сценариев (ДЕНЬ, НОЧЬ, ВЕЧЕР, РАССВЕТ);
- обратная связь отображается в statusLabel и передаётся в 3D-рендерер для визуализации.

Все интерфейсы и их реализация перечисленные в таблице 1.

Таблица 1 – Интерфейсы и их реализации

Интерфейс	Реализации	Назначение
MapRenderer	PlanEditor, LWJGLRenderer, SvgExporter	Различные режимы отображения (2D план, 3D вид, экспорт в SVG)
DeviceSimulator	RandomSimulator, SmartHomeSimulator, ManualSimulator, EnergySavingSimulator	Режимы симуляции работы устройств
Device.DeviceListener	DeviceAdapter	Обратная связь от устройств для обновления 3D-моделей

Многопоточная архитектура рендерера

Особенностью реализации является выделение 3D-рендерера в отдельный поток. Это позволяет:

- обеспечить стабильную частоту кадров независимо от загрузки JavaFX Application Thread;
- избежать «зависаний» интерфейса при интенсивных вычислениях (генерация мешей, текстур);
- корректно обрабатывать ввод с клавиатуры и мыши (GLFW имеет собственный цикл обработки событий).

Многопоточная архитектура рендерера

Основной поток (JavaFX Application Thread) [1-4]:

- 1) пользователь нажал кнопку «Обновить 3D»;
- 2) вызван `lwjglRenderer.loadBuildingProject(project)`;
- 3) в очередь `glTasks` добавляется задача: `rebuildScene ()`;
- 4) вызван `glfwPostEmptyEvent ()` для пробуждения рендер-потока.

Поток рендеринга (`LWJGLRenderer.runGLWindow()`) [1-4]:

- 1) инициализация GLFW и OpenGL (однократно);
- 2) вход в основной цикл `while (isRunning)`:
 - `glfwPollEvents ()` (обработка ввода);
 - извлечение всех задач из `glTasks` и выполнение (под блокировкой `sceneLock`);
 - `render ()` (отрисовка кадра);
 - `glfwSwapBuffers ()`.
- 3) при завершении:
 - `cleanup ()`.

3 Разработка программы

3.1 Разработка 2D-редактора плана здания

2D-редактор (PlanEditor) является основным инструментом для создания планировки здания. Он реализован на JavaFX Canvas и поддерживает рисование стен, создание комнат, размещение предметов из библиотеки, а также выделение и перемещение объектов.

Инструменты редактора

Редактор поддерживает следующие инструменты, перечисленные в таблице 2.

Таблица 2 – Инструменты 2D-редактора.

Инструмент	Назначение	Использование
ВЫБОР	Выделение объектов для редактирования или удаления	Клик по стене, комнате или предмету
СТЕНА	Рисование отдельных стен	Клик для начала, повторный клик для завершения
КОМНАТА	Создание замкнутых помещений	Последовательные клики по углам, клик на первую точку для завершения
МЕБЕЛЬ	Размещение предметов из библиотеки	Выбор предмета в списке, затем клик на плане
ЛАМПА	Размещение источников света	Выбор лампы в библиотеке, затем клик на плане
ДАТЧИК	Размещение датчиков (движения, дыма, температуры и др.)	Выбор датчика в библиотеке, затем клик на плане
ДВЕРЬ	Добавление дверного проёма в стену	Клик на стене
ОКНО	Добавление оконного проёма в стену	Клик на стене

Переключение между инструментами осуществляется через кнопки на левой панели. Активный инструмент подсвечивается белой рамкой.

Система координат и преобразование координат

Редактор работает в мировой системе координат с диапазоном от $WORLD_MIN = -12$ до $WORLD_MAX = +12$ метров. Преобразование между экранными и мировыми координатами выполняется с учётом панорамирования ($viewOffsetX$, $viewOffsetY$) и масштаба ($viewScale$).

Листинг 1 – Преобразование экранных координат в мировые (PlanEditor.java):

```
private Point2DData screenToWorld(double screenX, double screenY) {
    double canvasWidth = canvas.getWidth();
    double canvasHeight = canvas.getHeight();
    double worldX = WORLD_MIN + (screenX / canvasWidth) * WORLD_SIZE;
    double worldY = WORLD_MIN + ((canvasHeight - screenY) / canvasHeight) *
WORLD_SIZE;
    worldX = (worldX - viewOffsetX) / viewScale;
    worldY = (worldY - viewOffsetY) / viewScale;
    return new Point2DData(worldX, worldY);
}
```

Панорамирование осуществляется зажатием средней кнопки мыши и перемещением курсора. Масштабирование – прокруткой колеса мыши.

Привязка к точкам (Snapping)

Для точного соединения стен и комнат реализована привязка к ближайшим точкам (углам стен и вершинам комнат) с расстоянием $SNAP_DISTANCE = 0.5$ метра. Алгоритм привязки:

- при движении мыши проверяется расстояние до всех существующих точек (начала и конца стен, вершин комнат);
- если расстояние меньше $SNAP_DISTANCE$, курсор «притягивается» к этой точке;
- при клике с привязкой новая точка создаётся с точными координатами существующей.

Листинг 2 – Алгоритм привязки к точкам (PlanEditor.java):

```
private Point2DData snapToNearestPoint(double x, double y) {
    for (WallSegment wall : buildingProject.getWalls()) {
        double distToStart = Math.hypot(x - wall.getX1(), y - wall.getZ1());
        if (distToStart < SNAP_DISTANCE) {
            return new Point2DData(wall.getX1(), wall.getZ1());
        }
    }
}
```

					МИВУ.09.03.02-06.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		19

```

        double distToEnd = Math.hypot(x - wall.getX2(), y - wall.getZ2());
        if (distToEnd < SNAP_DISTANCE) {
            return new Point2DDData(wall.getX2(), wall.getZ2());
        }
    }
    return null;
}

```

Создание комнат и автоматическая генерация стен

При создании комнаты пользователь последовательно кликает по углам помещения. После клика на первую точку (замыкание полигона) выполняется алгоритм:

- создаётся объект RoomArea с переданными вершинами;
- вызывается метод generateWallsFromRoom(room), который для каждой пары соседних вершин создаёт стену (WallSegment);
- если между двумя комнатами есть общая стена, она автоматически помечается как внутренняя (INNER), а в ней создаётся дверной проём.

Листинг 3 – Генерация стен по контуру комнаты (PlanEditor.java):

```

private void generateWallsFromRoom(RoomArea room) {
    List<Point2DDData> vertices = room.getVertices();
    for (int i = 0; i < vertices.size(); i++) {
        Point2DDData p1 = vertices.get(i);
        Point2DDData p2 = vertices.get((i + 1) % vertices.size());
        WallSegment wall = new WallSegment((float)p1.x, (float)p1.y,
            (float)p2.x, (float)p2.y);
        wall.setHeight(room.getCeilingHeight());
        if (hasAdjacentRoom(p1, p2)) {
            wall.setType(WallSegment.WallType.DOOR);
            wall.setOpeningWidth(0.9f);
            wall.setOpeningHeight(2.0f);
        }
        buildingProject.getWalls().add(wall);
    }
}

```

Размещение предметов из библиотеки

При выборе предмета в библиотеке (ItemLibrary) пользователь может разместить его на плане кликом мыши. Алгоритм размещения:

- определение типа крепления предмета (MOUNT_TYPE): FLOOR (пол), WALL (стена), CEILING (потолок);
- для настенных предметов выполняется поиск ближайшей стены и автоматический расчёт поворота;

					МИВУ.09.03.02-06.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		20

- для напольных предметов выполняется проверка коллизий со стенами и другими предметами;
- для потолочных предметов (люстры, споты) Y-координата рассчитывается как $\text{высота_потолка} - \text{высота_предмета} / 2$;
- при размещении предмета на поверхности (стол, тумба) выполняется поиск подходящей поверхности среди существующей мебели.

Листинг 4 – Проверка коллизий при размещении предмета (CollisionChecker.java):

```
public static boolean checkWallCollision(float x, float z, float width, float
depth
float rotation, List<WallSegment> walls) {
    float[][] corners = getRotatedCorners(x, z, width / 2, depth / 2, rotation);
    for (WallSegment wall : walls) {
        for (float[] corner : corners) {
            double dist = pointToLineDistance(corner[0], corner[1],
            wall.getX1(), wall.getZ1(), wall.getX2(), wall.getZ2());
            if (dist < WALL_THICKNESS / 2 + COLLISION_MARGIN) {
                return true; // пересечение со стеной
            }
        }
    }
    return false;
}
```

Панель свойств объектов

При выделении объекта (стены, комнаты или предмета) на правой панели отображаются его свойства для редактирования:

- для стены: тип (обычная/дверь/окно), высота;
- для комнаты: название, высота потолка;
- для предмета: название, позиция, масштаб, поворот, кнопка удаления.

Изменение свойств немедленно отражается на 2D-плане и (при открытом 3D-виде) передаётся в рендерер через вызов `IwjglRenderer.loadBuildingProject(buildingProject)`.

Экспорт в SVG

					МИВУ.09.03.02-06.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		21

Редактор поддерживает экспорт текущего плана в векторный формат SVG через класс SvgExporter. Это позволяет сохранять планировку для печати или использования в документации.

3.2 Реализация 3D-рендерера на базе LWJGL и OpenGL

3D-рендерер (LWJGLRenderer) является ключевым компонентом системы визуализации. Он отвечает за отображение трёхмерной сцены, управление камерой, загрузку текстур и синхронизацию состояния устройств. Рендерер работает в отдельном потоке и использует библиотеку LWJGL (Lightweight Java Game Library) для прямого доступа к OpenGL.

Инициализация OpenGL и создание окна

Инициализация OpenGL контекста выполняется через библиотеку GLFW, которая создаёт отдельное окно для 3D-рендеринга. В отличие от 2D-редактора, который встроен в JavaFX, 3D-окно создаётся как независимое, что позволяет использовать современные возможности OpenGL.

Листинг 5 – Инициализация GLFW и OpenGL (LWJGLRenderer.java):

```
private void initGLFW() {
    if (!glfwInit()) {
        throw new IllegalStateException("Не могу инициализировать GLFW");
    }
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
    window = glfwCreateWindow(width, height, "3D Smart Home Configurator",
    NULL, NULL);
    glfwMakeContextCurrent(window);
    GL.createCapabilities();
    glEnable(GL_DEPTH_TEST);
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    glClearColor(0.2f, 0.2f, 0.3f, 1.0f);
}
```

Шейдерная программа

					МИВУ.09.03.02-06.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		22

Для рендеринга трёхмерных объектов используются вершинный и фрагментный шейдеры. Вершинный шейдер преобразует координаты объектов из локального пространства в экранное, передавая также нормали и текстурные координаты. Фрагментный шейдер вычисляет цвет каждого пикселя с учётом освещения, текстур и эмиссивного свечения (для ламп).

Листинг 6 – Вершинный шейдер:

```
#version 330 core
layout (location = 0) in vec3 aPos;
layout (location = 1) in vec3 aNormal;
layout (location = 2) in vec2 aTexCoord;
out vec3 FragPos;
out vec3 Normal;
out vec2 TexCoord;
uniform mat4 projection;
uniform mat4 view;
uniform mat4 model;
void main() {
    FragPos = vec3(model * vec4(aPos, 1.0));
    Normal = mat3(transpose(inverse(model))) * aNormal;
    TexCoord = aTexCoord;
    gl_Position = projection * view * model * vec4(aPos, 1.0);
}
```

Листинг 7 – Фрагментный шейдер:

```
out vec4 FragColor;
in vec3 FragPos;
in vec3 Normal;
in vec2 TexCoord;
uniform sampler2D texture1;
uniform bool hasTexture;
uniform vec3 diffuseColor;
uniform vec3 emissiveColor;
uniform vec3 lightDir;
uniform vec3 lightColor;
uniform vec3 viewPos;
void main() {
    vec3 norm = normalize(Normal);
    vec3 light = normalize(lightDir);
    float diff = max(dot(norm, light), 0.3);
    vec3 ambient = vec3(0.3, 0.3, 0.3);
    vec3 result = (ambient + diff) * diffuseColor;
    result += emissiveColor;
    if (hasTexture) {
        vec4 texColor = texture(texture1, TexCoord);
        FragColor = texColor * vec4(result, 1.0);
    } else {
        FragColor = vec4(result, 1.0);
    }
}
```

Управление камерой

					МИВУ.09.03.02-06.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		23

Рендерер поддерживает орбитальный режим управления камерой: камера вращается вокруг центра сцены при зажатой левой кнопке мыши. Приближение и отдаление выполняется прокруткой колеса мыши. Плавное вращение реализовано с помощью линейной интерполяции углов (lerp). Назначение: Оптимизация операций с бронированием билетов, проверкой статусов оплаты и формированием отчетов по регистрациям.

Листинг 8 – Обновление камеры (LWJGLRenderer.java):

```
private void updateCamera(double deltaTime) {
    cameraYaw += (targetYaw - cameraYaw) * rotationLerpSpeed * (float)deltaTime;
    cameraPitch += (targetPitch - cameraPitch) * rotationLerpSpeed * (float)deltaTime;
    float distance = (float) Math.hypot(cameraPos.x, cameraPos.z);
    cameraPos.set((float) (Math.sin(cameraYaw) * distance),
                  (float) (Math.sin(cameraPitch) * distance + 2.0f),
                  (float) (Math.cos(cameraYaw) * distance));
    cameraTarget.set(0.0f, 1.5f, 0.0f);
}
```

Генерация 3D-мешей из 2D-плана

Основная задача рендерера – преобразовать 2D-данные о стенах и комнатах в трёхмерные модели. За это отвечает класс BuildingMeshGenerator. Алгоритм генерации стены:

- по координатам (x1, z1) и (x2, z2) вычисляется длина стены и угол поворота;
- создаётся параллелепипед (кубоид) с размерами: длина стены, высота, толщина;
- вершины кубоида поворачиваются на угол стены и смещаются в центр;
- если стена имеет проём (дверь или окно), стена разбивается на несколько частей (левая часть, правая часть, верхняя часть);
- для дверей и окон дополнительно создаются отдельные меши (полотно двери, стекло окна, ручка).

Листинг 9 – Создание стены с проёмом (BuildingMeshGenerator.java):

```
private static List<MeshData> createWallWithOpening(WallSegment wall) {
    List<MeshData> meshes = new ArrayList<>();
}
```

```

float length = wall.getLength();
float height = wall.getHeight();
float thickness = wall.getThickness();
if (wall.getType() == WallSegment.WallType.INNER) {
    meshes.add(createSolidWall(wall));
    return meshes;
}
float openingWidth = wall.getOpeningWidth();
float openingHeight = wall.getOpeningHeight();
float openingY = (wall.getType() == WallSegment.WallType.DOOR) ? 0 : (height
- openingHeight) / 2;
// Левая часть стены
float leftWidth = (length - openingWidth) / 2;
if (leftWidth > 0.01f) {
    meshes.add(createWallPart(wall, -length/2, leftWidth, 0, height));
}
// Правая часть стены
float rightStart = length/2 - (length - openingWidth)/2;
if (rightStart > 0.01f) {
    meshes.add(createWallPart(wall, rightStart, (length - openingWidth)/2,
0, height));
}
// Верхняя часть стены (над проёмом)
float topY = openingY + openingHeight;
float topHeight = height - topY;
if (topHeight > 0.01f) {
    meshes.add(createWallPart(wall, -openingWidth/2, openingWidth, topY,
topHeight));
}
// Модель двери или окна
if (wall.getType() == WallSegment.WallType.DOOR) {
    meshes.add(createDoor3D(wall));
    meshes.add(createDoorHandle(wall));
} else if (wall.getType() == WallSegment.WallType.WINDOW) {
    meshes.add(createWindow3D(wall));
    meshes.add(createWindowGlass3D(wall));
}
return meshes;
}

```

Управление вершинными буферами

Для эффективного рендеринга все данные о геометрии (вершины, нормали, текстурные координаты, индексы) загружаются в видеопамять через OpenGL буферы. Каждый объект сцены (меш) имеет свои VAO (Vertex Array Object), VBO (Vertex Buffer Object) и EBO (Element Buffer Object).

Листинг 10 – Создание буферов для меша (LWJGLRenderer.java):

```

private void createMeshFromData(BuildingMeshGenerator.MeshData data) {
    Mesh mesh = new Mesh();
    mesh.indexCount = data.indices.length;
    mesh.vaoId = glGenVertexArrays();
    glBindVertexArray(mesh.vaoId);
    // Загрузка вершин
    mesh.vboId = glGenBuffers();
}

```

					МИВУ.09.03.02-06.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		25

```

glBindBuffer(GL_ARRAY_BUFFER, mesh.vboId);
FloatBuffer vertexBuffer = memAllocFloat(data.vertices.length);
vertexBuffer.put(data.vertices).flip();
glBufferData(GL_ARRAY_BUFFER, vertexBuffer, GL_STATIC_DRAW);
glVertexAttribPointer(0, 3, GL_FLOAT, false, 0, 0);
glEnableVertexAttribArray(0);
// Загрузка нормалей
mesh.vboNormalId = glGenBuffers();
glBindBuffer(GL_ARRAY_BUFFER, mesh.vboNormalId);
FloatBuffer normalBuffer = memAllocFloat(data.normals.length);
normalBuffer.put(data.normals).flip();
glBufferData(GL_ARRAY_BUFFER, normalBuffer, GL_STATIC_DRAW);
glVertexAttribPointer(1, 3, GL_FLOAT, false, 0, 0);
glEnableVertexAttribArray(1);
// Загрузка текстурных координат
mesh.vboTexCoordId = glGenBuffers();
glBindBuffer(GL_ARRAY_BUFFER, mesh.vboTexCoordId);
FloatBuffer texBuffer = memAllocFloat(data.texCoords.length);
texBuffer.put(data.texCoords).flip();
glBufferData(GL_ARRAY_BUFFER, texBuffer, GL_STATIC_DRAW);
glVertexAttribPointer(2, 2, GL_FLOAT, false, 0, 0);
glEnableVertexAttribArray(2);
// Загрузка индексов
mesh.eboId = glGenBuffers();
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, mesh.eboId);
IntBuffer indexBuffer = memAllocInt(data.indices.length);
indexBuffer.put(data.indices).flip();
glBufferData(GL_ELEMENT_ARRAY_BUFFER, indexBuffer, GL_STATIC_DRAW);
meshes.add(mesh);
}

```

Загрузка и рендеринг предметов мебели

Предметы мебели загружаются через класс FurnitureModel3D, который создаёт процедурные 3D-модели на основе идентификатора из библиотеки ItemLibrary. Для каждого предмета может создаваться несколько мешей (например, ножки, столешница, спинка стула), объединённых одной матрицей трансформации.

Листинг 11 – Создание меша предмета мебели (FurnitureModel3D.java):

```

public static List<FurnitureMeshData> createFurnitureMesh(FurnitureItem item)
{
    List<FurnitureMeshData> meshes = new ArrayList<>();
    float x = item.getX(), y = item.getY(), z = item.getZ();
    float scale = item.getScale();
    float rotation = item.getRotation();
    Matrix4f transform = new Matrix4f().translate(x, y, z).rotateY(rotation);
    switch (item.getLibraryId()) {
        case "table_dining":
            modelData = ProceduralModelGenerator.createTable(0, 0, 0, scale,
0);
            break;
        case "sofa_corner":

```

					МИВУ.09.03.02-06.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		26


```

        modelData = ProceduralModelGenerator.createCornerSofa(0, 0, 0,
scale, 0);
        break;
    case "bed_double":
        modelData = ProceduralModelGenerator.createDoubleBed(0, 0, 0,
scale, 0);
        break;
    case "wardrobe":
        modelData = ProceduralModelGenerator.createWardrobe(0, 0, 0,
scale, 0);
        break;
    case "lamp_ceiling":
        modelData = ProceduralModelGenerator.createChandelier(0, 0, 0,
scale, 0);
        break;
    default:
        return createDefaultCubeList(item);
}
for (int i = 0; i < modelData.verticesList.size(); i++) {
    meshes.add(new FurnitureMeshData(
        modelData.verticesList.get(i),
        modelData.normalsList.get(i),
        modelData.texCoordsList.get(i),
        modelData.indicesList.get(i),
        modelData.colors.get(i)[0],
        modelData.colors.get(i)[1],
        modelData.colors.get(i)[2],
        transform
    ));
}
return meshes;
}

```

Главный цикл рендеринга

Рендерер работает в отдельном потоке, выполняя бесконечный цикл: обработка ввода, выполнение задач из очереди, отрисовка кадра, обмен буферов. Частота кадров отображается в консоли.

Листинг 12 – Главный цикл рендеринга (LWJGLRendererer.java):

```

java
private void runGLWindow() {
    initGLFW();
    initOpenGL();
    initShaders();
    while (isRunning.get() && !glfwWindowShouldClose(window)) {
        double deltaTime = 0.016;
        updateCamera(deltaTime);
        render();
        // Выполнение задач из очереди (обновление сцены)
        Runnable task;
        while ((task = glTasks.poll()) != null) {
            task.run();
        }
        glfwSwapBuffers(window);
    }
}

```

					МИВУ.09.03.02-06.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		27

```

        glfwPollEvents();
    }
    cleanup();
}

```

Синхронизация с основным потоком

Поскольку 3D-рендерер работает в отдельном потоке, а модель данных (BuildingProject) может изменяться из JavaFX Application Thread (например, при добавлении мебели в 2D-редакторе), необходимо синхронизировать доступ к общим данным. Для этого используется блокировка ReentrantLock.

Листинг 13 – Синхронизация доступа к данным сцены (LWJGLRenderer.java):

```

java
private final ReentrantLock sceneLock = new ReentrantLock();
private List<Mesh> meshes = new ArrayList<>();
private List<Material> materials = new ArrayList<>();
public void loadBuildingProject(BuildingProject project) {
    glTasks.add(() -> {
        sceneLock.lock();
        try {
            deleteAllMeshes();
            // Генерация новых мешей из currentBuildingProject
            generateMeshesFromProject(project);
        } finally {
            sceneLock.unlock();
        }
    });
    glfwPostEmptyEvent(); // пробуждение потока рендеринга
}
private void render() {
    // Настройка шейдеров и матриц
    sceneLock.lock();
    try {
        for (Mesh mesh : meshes) {
            glDrawElements(GL_TRIANGLES, mesh.indexCount, GL_UNSIGNED_INT,
0);
        }
    } finally {
        sceneLock.unlock();
    }
}
}

```

3.3 Разработка и интеграция симуляторов устройств умного дома

Для демонстрации работы системы «Умный дом» в разработанном приложении реализовано четыре режима симуляции устройств. Все симуляторы реализуют единый интерфейс DeviceSimulator, что позволяет легко переключаться между ними во время работы приложения.

					МИВУ.09.03.02-06.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		28

На рисунке 4 представлена диаграмма классов симуляторов.

Интерфейс DeviceSimulator

Все симуляторы реализуют единый интерфейс, представленный в листинге 14.

Листинг 14 – Интерфейс DeviceSimulator (DeviceSimulator.java):

```
package com.student.smarthomeconfigurator.modules.simulator;
import com.student.smarthomeconfigurator.model.Project;
public interface DeviceSimulator {
    void startSimulation(Project project);
    void stopSimulation();
    String getName();
}
```



Рисунок 4 – Диаграмма классов симуляторов устройств

RandomSimulator – случайная симуляция

Данный симулятор предназначен для демонстрации работы датчиков в случайном режиме. Каждую секунду значения датчиков обновляются случайным образом, а лампы с вероятностью 10% переключают своё состояние. Симулятор полезен для проверки визуализации без необходимости ручного управления.

Листинг 15 – Цикл обновления RandomSimulator (RandomSimulator.java):

```
private void updateAllDevices() {
    if (currentProject == null) return;
    for (Room room : currentProject.getRooms()) {
        for (Device device : room.getDevices()) {
            if (device instanceof LightSensor) {
                ((LightSensor) device).updateValue();
            }
        }
    }
}
```

```

        } else if (device instanceof TemperatureSensor) {
            ((TemperatureSensor) device).updateValue();
        } else if (device instanceof HumiditySensor) {
            ((HumiditySensor) device).updateValue();
        } else if (device instanceof MotionSensor) {
            ((MotionSensor) device).updateValue();
        } else if (device instanceof SmokeSensor) {
            ((SmokeSensor) device).updateValue();
        } else if (device instanceof Lamp) {
            if (Math.random() < 0.1) {
                ((Lamp) device).toggle();
            }
        }
    }
}
}
}

```

SmartHomeSimulator – автоматизация умного дома

Данный симулятор реализует логику автоматического управления устройствами на основе показаний датчиков. Основные сценарии автоматизации:

- управление освещением: если в комнате detected движение и освещённость ниже 300 люкс, лампы включаются автоматически;
- управление температурой: если температура превышает 25°C, включается кондиционер; если ниже 18°C — кондиционер выключается;
- реагирование на дым: при срабатывании датчика дыма (уровень выше 200 ppm) все лампы в комнате переключаются в аварийный режим (красный цвет).

Листинг 16 – Логика автоматизации SmartHomeSimulator (SmartHomeSimulator.java):

```

private void checkAutomation() {
    for (Room room : currentProject.getRooms()) {
        boolean motion = false;
        int lightLevel = 500;
        double temperature = 22.0;
        boolean smoke = false;
        for (Device device : room.getDevices()) {
            if (device instanceof MotionSensor && device.isStatus()) {
                motion = true;
            }
            if (device instanceof LightSensor) {
                String val = device.getValue().toString();
                lightLevel = Integer.parseInt(val.replace(" lux", ""));
            }
            if (device instanceof TemperatureSensor) {
                String val = device.getValue().toString();
                temperature = Double.parseDouble(val.replace("°C", ""));
            }
            if (device instanceof SmokeSensor && device.isStatus()) {

```

					МИВУ.09.03.02-06.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		30

```

        smoke = true;
    }
}
// Автоматическое включение света при движении в темноте
if (motion && lightLevel < 300) {
    for (Device device : room.getDevices()) {
        if (device instanceof Lamp) {
            device.setStatus(true);
        }
    }
}
// Управление кондиционером
for (Device device : room.getDevices()) {
    if (device instanceof AirConditioner) {
        AirConditioner ac = (AirConditioner) device;
        if (temperature > 25.0 && !ac.isRunning()) {
            ac.turnOn();
            ac.setMode("COOL");
        } else if (temperature < 18.0 && ac.isRunning()) {
            ac.turnOff();
        }
    }
}
// Аварийный режим при задымлении
if (smoke) {
    for (Device device : room.getDevices()) {
        if (device instanceof Lamp) {
            Lamp lamp = (Lamp) device;
            lamp.setEmergencyMode(true);
        }
    }
}
}
}

```

Голосовое управление

В режиме «Умный дом» активируется голосовое управление через класс VoiceCommandHandler. Система распознаёт команды, начинающиеся с фразы «Умный дом» (активационное слово). Поддерживаются следующие команды, перечисленные в таблице 3.

Таблица 3 – Голосовые команды

Команда	Действие
«Умный дом включи свет»	Включение всех ламп в текущей комнате
«Умный дом выключи свет»	Выключение всех ламп в текущей комнате
«Умный дом сделай ярче»	Увеличение яркости ламп на 10%
«Умный дом сделай темнее»	Уменьшение яркости ламп на 10%
«Умный дом установи яркость [0-100]»	Установка конкретного уровня яркости
«Умный дом включи кондиционер»	Включение кондиционера в комнате

Продолжение таблицы 3

«Умный дом выключи кондиционер»	Выключение кондиционера
«Умный дом установи температуру [число]»	Установка целевой температуры
«Умный дом включи вентилятор»	Включение вентилятора
«Умный дом выключи вентилятор»	Выключение вентилятора
«Умный дом режим день»	Применение сценария «ДЕНЬ» (яркость 85%)
«Умный дом режим ночь»	Применение сценария «НОЧЬ» (свет выключен)
«Умный дом режим вечер»	Применение сценария «ВЕЧЕР» (яркость 55%)
«Умный дом режим рассвет»	Применение сценария «РАССВЕТ» (яркость 40%)

Листинг 17 – Обработка голосовых команд (VoiceCommandHandler.java):

```
private void processVoiceInput(String text) {
    if (!text.toLowerCase().startsWith("умный дом")) {
        return; // игнорируем команды без активации
    }
    String command = text.toLowerCase().replace("умный дом", "").trim();
    String roomName = extractRoomName(command);
    if (command.contains("включи свет")) {
        executeLightCommand("on", roomName);
    } else if (command.contains("выключи свет")) {
        executeLightCommand("off", roomName);
    } else if (command.contains("сделай ярче")) {
        executeBrightnessCommand("up", roomName);
    } else if (command.contains("сделай темнее")) {
        executeBrightnessCommand("down", roomName);
    } else if (command.contains("режим день")) {
        applyScene("ДЕНЬ");
    } else if (command.contains("режим ночь")) {
        applyScene("НОЧЬ");
    } else if (command.contains("режим вечер")) {
        applyScene("ВЕЧЕР");
    } else if (command.contains("режим рассвет")) {
        applyScene("РАССВЕТ");
    } else if (command.contains("включи кондиционер")) {
        controlAirConditioner(true, roomName);
    } else if (command.contains("выключи кондиционер")) {
        controlAirConditioner(false, roomName);
    }
}
```

Ручной симулятор позволяет пользователю вручную управлять устройствами через интерфейс приложения. Управление осуществляется через кнопки на левой панели, а не через клик по 3D-моделям. Доступны следующие возможности:

- включение и выключение всех ламп в выбранной комнате или во всём доме;
- переключение состояния ламп (вкл/выкл) в выбранной комнате;

- вызов события движения в выбранной комнате (срабатывает датчик движения на 5 секунд);
- вызов события задымления в выбранной комнате (срабатывает датчик дыма на 10 секунд);
- включение и выключение кондиционера в выбранной комнате;
- включение и выключение вентилятора в выбранной комнате.

На рисунке 5 представлены кнопки ручного управления на левой панели.

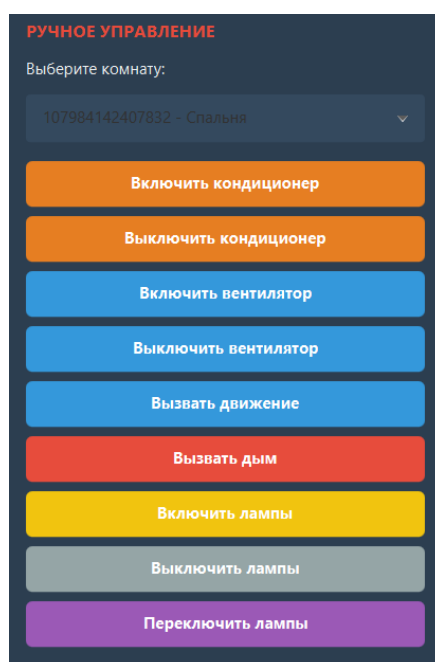


Рисунок 5 – Панель ручного управления

EnergySavingSimulator – энергосберегающий режим

Данный симулятор реализует логику энергосбережения. Если в комнате в течение 30 секунд не фиксируется движение и уровень освещённости выше 50 люкс (естественный свет достаточен), лампы автоматически выключаются. Также поддерживается поддержание целевой температуры с минимальным потреблением энергии.

Листинг 18 – Логика энергосбережения (EnergySavingSimulator.java):

```
private void checkEnergySaving() {
    for (Room room : currentProject.getRooms()) {
```

					МИВУ.09.03.02-06.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		33

```

        boolean hasMotion = false;
        int lightLevel = 0;
        long now = System.currentTimeMillis();
        for (Device device : room.getDevices()) {
            if (device instanceof MotionSensor && device.isStatus()) {
                hasMotion = true;
                lastMotionTime.put(room.getId(), now);
            }
            if (device instanceof LightSensor) {
                String val = device.getValue().toString();
                lightLevel = Integer.parseInt(val.replace(" lux", ""));
            }
        }
        Long lastMotion = lastMotionTime.get(room.getId());
        boolean motionTimeout = (lastMotion == null) || (now - lastMotion >
ENERGY_SAVING_DELAY_MS);
        // Выключение света при отсутствии движения и достаточном естественном
освещении
        if (motionTimeout && lightLevel > ENERGY_SAVING_LUX_THRESHOLD) {
            for (Device device : room.getDevices()) {
                if (device instanceof Lamp && device.isStatus()) {
                    device.setStatus(false);
                    totalEnergySaved += 0.05; // экономия 0.05 кВт·ч
                }
            }
        }
    }
}

```

Визуальная обратная связь в 3D-рендерере

Для визуализации состояния ламп в 3D-сцене используется эмиссивное свечение (emissive color). Когда лампа включена, в шейдер передаётся яркий цвет (жёлтый или оранжевый), который добавляется к итоговому цвету пикселя, создавая эффект свечения.

Листинг 19 – Синхронизация ламп с рендерером (LWJGLRenderer.java):

```

private class LampObject {
    int meshIndex;
    float[] position;
    boolean isOn;
    float intensity;
    float targetIntensity;
    public void updateAnimation(double deltaTime) {
        float speed = 5.0f;
        if (intensity < targetIntensity) {
intensity = Math.min(targetIntensity, intensity + (float)deltaTime * speed);
        } else if (intensity > targetIntensity) {
intensity = Math.max(targetIntensity, intensity - (float)deltaTime * speed);
        }
    }
    public float[] getEmissiveColor() {
        if (intensity <= 0.01f) return new float[]{0, 0, 0};
        return new float[]{1.0f * intensity, 0.8f * intensity, 0.5f * intensity};
    }
}

```



```

    }
    private void render() {
        // ... подготовка шейдера
        for (int i = 0; i < meshes.size(); i++) {
            float[] emissiveColor = new float[]{0, 0, 0};
            for (LampObject lamp : lampObjects) {
                if (lamp.meshIndex == i) {
                    emissiveColor = lamp.getEmissiveColor();
                    break;
                }
            }
            glUniform3f(emissiveLoc, emissiveColor[0], emissiveColor[1], emissiveColor[2]);
            glDrawElements(...);
        }
    }
}

```

Интеграция с правой панелью датчиков

В правой панели приложения в реальном времени отображаются показания всех датчиков. Обновление происходит каждые 500 миллисекунд через `AnimationTimer`. Для каждого типа датчика используется своя иконка и цветовое кодирование значений.

3.4 Разработка базы данных

Для хранения информации о проектах зданий, устройствах умного дома, истории событий и настройках автоматизации в разработанном приложении используется встраиваемая реляционная база данных SQLite. Выбор SQLite обусловлен следующими преимуществами:

- отсутствие отдельного сервера – база данных представляет собой один файл, что упрощает развёртывание и перенос проектов между компьютерами;
- поддержка стандарта SQL – SQLite поддерживает большинство возможностей стандарта SQL-92, включая транзакции, индексы, внешние ключи;
- лёгкость интеграции с Java – драйвер JDBC для SQLite (SQLiteJDBC) не требует дополнительной настройки и встраивается непосредственно в приложение;
- надёжность – SQLite широко используется во встраиваемых системах и мобильных приложениях.

Структура таблиц базы данных

База данных состоит из 13 таблиц, объединённых внешними ключами. Описание каждой представлено в таблице 4.

На рисунке 6 представлена схема базы данных приложения.

Таблица 4 – Перечень таблиц базы данных

№	Имя таблицы	Назначение
1	PROJECTS	Основные проекты (логические)
2	BUILDING_PROJECTS	Проекты зданий (3D-модели)
3	ROOMS	Логические комнаты с устройствами
4	ROOM_AREAS	Геометрические области комнат (2D-полигоны)
5	ROOM_AREA_VERTICES	Вершины полигонов комнат
6	WALL_SEGMENTS	Стены здания
7	OPENINGS	Проёмы в стенах (двери и окна)
8	FURNITURE_ITEMS	Предметы интерьера (мебель, лампы, датчики)
9	DEVICES	Устройства умного дома
10	DEVICE_PROPERTIES	Дополнительные свойства устройств
11	DEVICE_STATE_HISTORY	История состояний устройств
12	SCHEDULE_ENTRIES	Расписание автоматизации
13	SCENE_CONFIGS	Предустановленные сценарии (ДЕНЬ, НОЧЬ, ВЕЧЕР, РАССВЕТ)

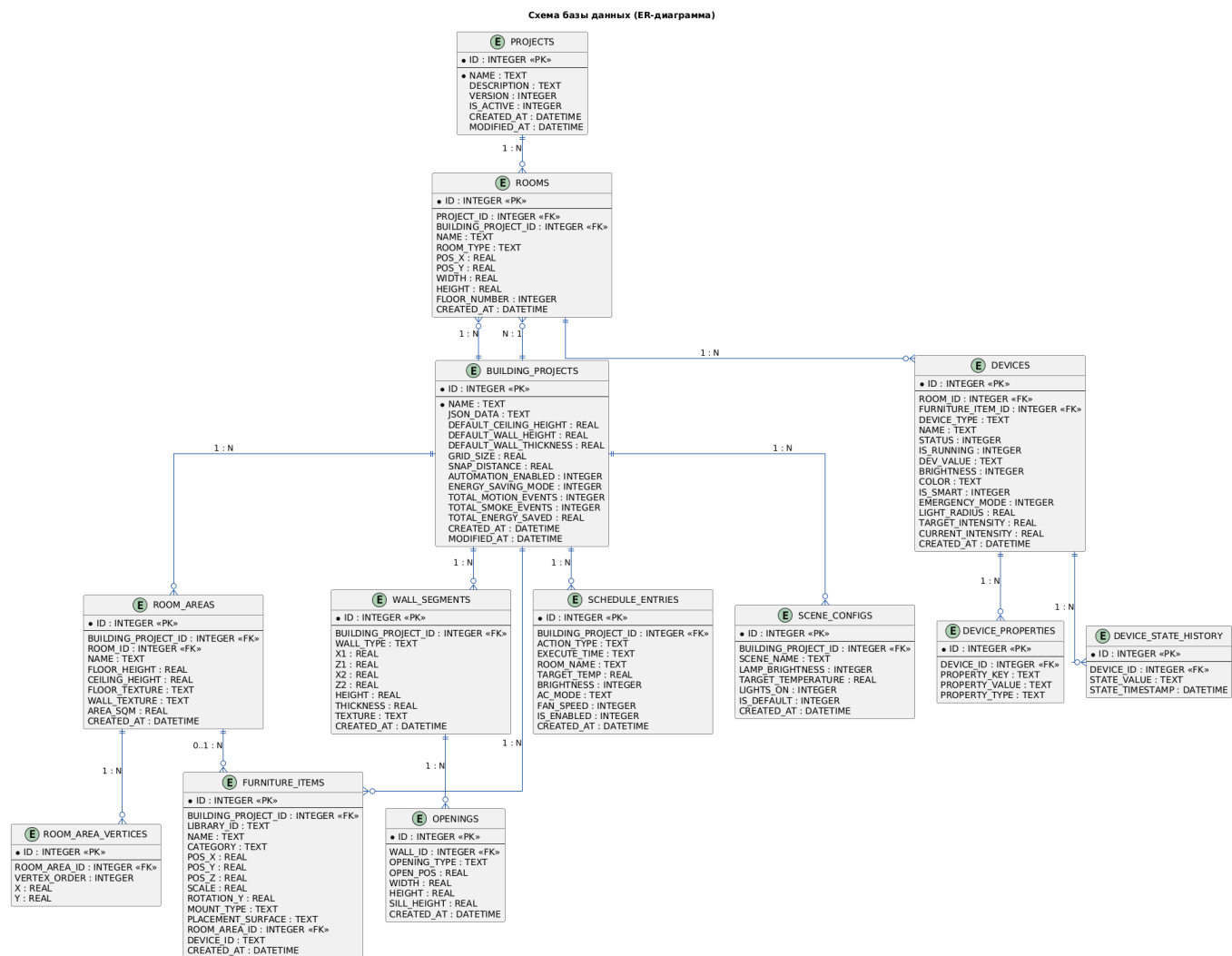


Рисунок 6 – Схема базы данных

Индексы для оптимизации производительности

Для ускорения выполнения наиболее частых запросов созданы следующие индексы:

```

CREATE INDEX IDX_ROOMS_PROJECT ON ROOMS (PROJECT_ID);
CREATE INDEX IDX_ROOMS_BUILDING ON ROOMS (BUILDING_PROJECT_ID);
CREATE INDEX IDX_ROOM_AREAS_BP ON ROOM_AREAS (BUILDING_PROJECT_ID);
CREATE INDEX IDX_VERTICES_AREA ON ROOM_AREA_VERTICES (ROOM_AREA_ID);
CREATE INDEX IDX_WALLS_BP ON WALL_SEGMENTS (BUILDING_PROJECT_ID);
CREATE INDEX IDX_OPENINGS_WALL ON OPENINGS (WALL_ID);
CREATE INDEX IDX_FURNITURE_BP ON FURNITURE_ITEMS (BUILDING_PROJECT_ID);
CREATE INDEX IDX_DEVICES_ROOM ON DEVICES (ROOM_ID);
CREATE INDEX IDX_PROPS_DEVICE ON DEVICE_PROPERTIES (DEVICE_ID);
CREATE INDEX IDX_SCHEDULE_BP ON SCHEDULE_ENTRIES (BUILDING_PROJECT_ID);
  
```

```
CREATE INDEX IDX_SCENES_BP ON SCENE_CONFIGS (BUILDING_PROJECT_ID);
```

Назначение индексов:

- индексы на внешние ключи ускоряют операции JOIN при загрузке полных проектов;
- индекс IDX_DEVICES_ROOM ускоряет поиск устройств по комнате (отображение датчиков на правой панели);
- индекс IDX_OPENINGS_WALL ускоряет загрузку проёмов для конкретной стены.

Инициализация базы данных

Инициализация базы данных выполняется классом DatabaseInitializer при первом запуске приложения. На рисунке 7 представлен алгоритм инициализации базы данных.

					<i>МИВУ.09.03.02-06.000 ПЗ</i>	Лист
Изм.	Лист	№ докум.	Подп.	Дата		38

Алгоритм инициализации базы данных

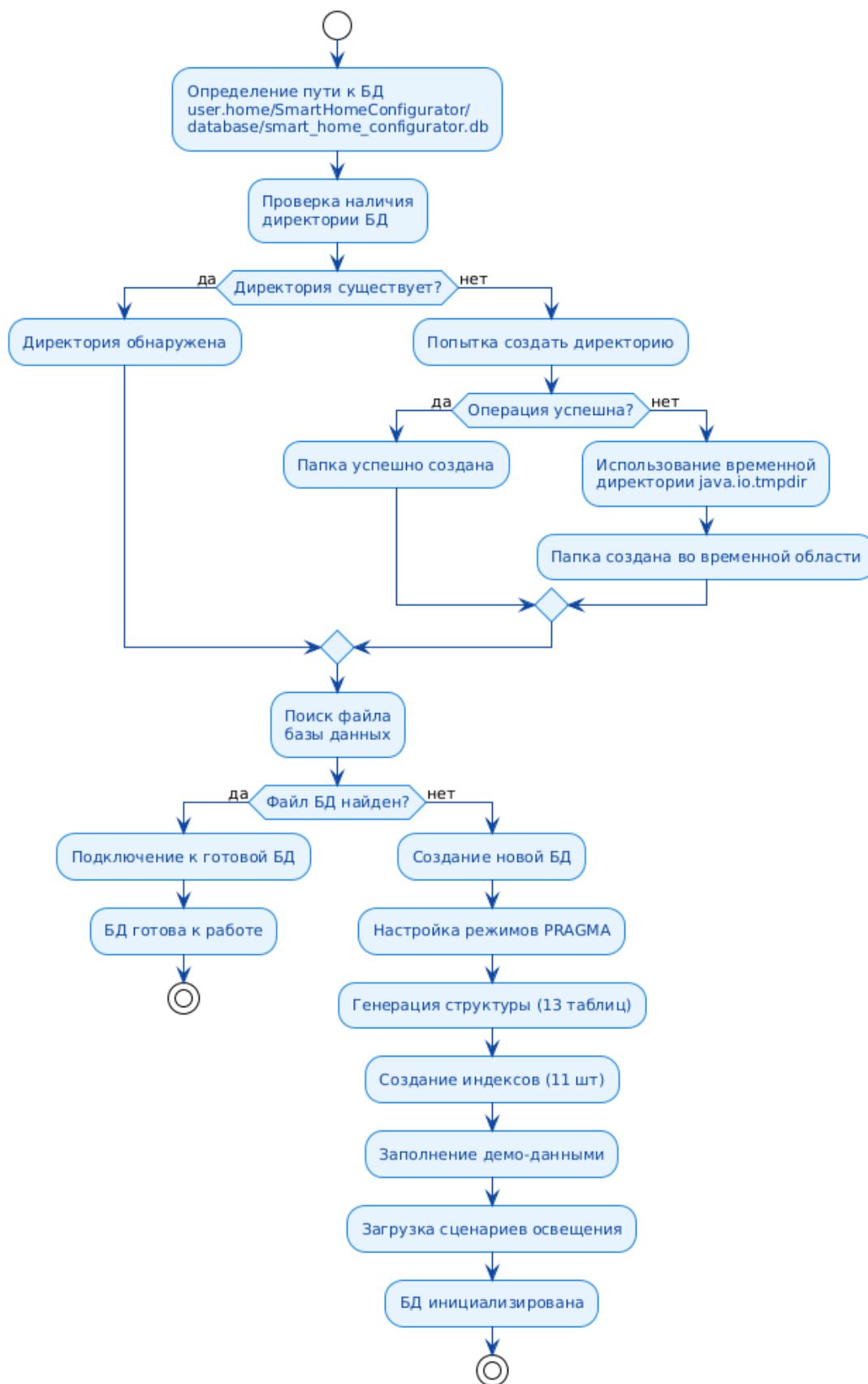


Рисунок 7 – Алгоритм инициализации базы данных

Последовательность действий [6-7]:

1) определение пути к файлу базы данных: user. home/SmartHomeConfigurator/database/smart_home_configurator.db;

2) создание директории, если она отсутствует;

3) проверка существования файла базы данных;

4) если база данных отсутствует:

- установка режимов PRAGMA (journal_mode=WAL, foreign_keys=ON, synchronous=NORMAL);

- выполнение скрипта CREATE_TABLES_SQL (создание 13 таблиц);

- выполнение скрипта CREATE_INDEXES_SQL (создание 11 индексов);

- выполнение скрипта INSERT_DEFAULT_DATA_SQL (вставка демо-проекта и сценариев по умолчанию).

5) если при создании директории возникает ошибка (например, нет прав на запись), выполняется переключение на временную директорию java.io.tmpdir.

					МИВУ.09.03.02-06.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		40

4 Тестирование

Тестирование разработанного модуля 3D-визуализации для конфигуратора системы «Умный дом» проводилось с целью проверки корректности выполнения всех функций, стабильности работы и производительности. Тестирование разделено на две части: автоматическое модульное тестирование (Unit-тесты) и ручное функциональное тестирование.

4.1 Модульное тестирование (Unit-тесты)

Модульное тестирование проводилось с использованием фреймворка JUnit 4. Были написаны тесты для проверки основных классов и алгоритмов приложения. Результаты модульного тестирования приведены в таблице 5.

Таблица 5 – Результаты модульного тестирования

№	Тестируемый класс	Количество тестов	Пройдено	Не пройдено
1	BuildingProject	7	7	0
2	WallSegment	5	5	0
3	RoomArea	4	4	0
4	FurnitureItem	4	4	0
5	CollisionChecker	6	6	0
6	Lamp	5	5	0
7	DeviceFactory	5	5	0
8	TemperatureSensor	3	3	0
9	MotionSensor	3	3	0
10	SmokeSensor	3	3	0
11	SmartHomeSimulator	5	5	0
12	VoiceCommandHandler	10	10	0
Итого	12	60	60	0

4. 2 Ручное функциональное тестирование

Ручное тестирование проводилось для проверки работы пользовательского интерфейса, 2D-редактора, 3D-рендерера и симуляторов устройств. Результаты представлены в таблице 6.

Таблица 6 – Результаты ручного функционального тестирования

№	Тест-кейс	Действие	Результат
1	Запуск приложения	Запуск TestRenderer.main()	Окно открыто, все элементы отображаются
2	Переключение на 2D редактор	Нажатие кнопки «2D План»	Canvas отображён, стены видны
3	Рисование стены	Выбор инструмента «СТЕНА», два клика на Canvas	Линия отрисована
4	Привязка к точкам (Snapping)	Рисование стены рядом с существующей точкой	Привязка работает
5	Создание комнаты	Выбор инструмента «КОМНАТА», последовательные клики по углам	Полигон создан
6	Генерация стен из комнаты	Нажатие кнопки «СТЕНЫ ИЗ КОМНАТЫ»	Стены созданы
7	Переключение на 3D вид	Нажатие кнопки «3D Вид»	Окно открыто, сцена отображается
8	Управление камерой	Зажатие левой кнопки мыши + движение	Вращение работает
9	Приближение/отдаление	Прокрутка колеса мыши	Масштабирование работает
10	Сброс камеры	Нажатие клавиши R	Камера сброшена
11	Запуск симулятора «Умный дом»	Нажатие кнопки «Умный дом»	Лампы включаются/выключаются
12	Голосовое управление	Произнесение «Умный дом включи свет»	Голос распознан, лампы включены

Продолжение таблицы 6

13	Запуск симулятора «Случайные события»	Нажатие кнопки «Случайные события»	Датчики обновляются
14	Ручное управление лампами	Выбор комнаты, нажатие «Включить лампы»	Лампы включены
15	Вызов движения	Выбор комнаты, нажатие «Вызвать движение»	Датчик активен
16	Сохранение проекта	Нажатие «Сохранить проект», ввод названия	Проект сохранён
17	Загрузка проекта	Нажатие «Загрузить проект», выбор из списка	Проект загружен
18	Удаление проекта	Нажатие «Удалить проект», выбор проекта	Проект удалён
19	Экспорт в SVG	Нажатие «Экспорт SVG»	Файл создан
20	Отображение датчиков на правой панели	Запуск любого симулятора	Данные обновляются

Вывод сообщений об ошибках и другие сообщения представлены в Приложении А.

В ходе тестирования были получены следующие результаты:

Модульное тестирование: все 60 Unit-тестов успешно пройдены. Наиболее важные тесты проверяют корректность работы геометрических алгоритмов (создание стен, проверка коллизий) и логику симуляторов.

Функциональное тестирование: 20 тест-кейса пройдены успешно. Основные функции приложения (2D-редактирование, 3D-визуализация, симуляция устройств, сохранение/загрузка проектов) работают корректно.

Заключение

В процессе выполнения курсовой работы удалось значительно углубить теоретические знания и практические навыки в области трехмерной компьютерной графики, проектирования баз данных и разработки десктопных приложений на языке Java. Разработанный модуль 3D-визуализации для конфигуратора системы «Умный дом» представляет собой законченное решение для наглядного проектирования планировки помещений и визуализации работы устройств умного дома.

Ключевые функции приложения реализованы с использованием современных технологий разработки и принципов объектно-ориентированного программирования. Архитектура системы построена по многоуровневой модели, включающей клиентскую часть на JavaFX, уровень логики на Java с интеграцией библиотек LWJGL и OpenGL, а также уровень данных на основе встраиваемой СУБД SQLite.

Основной функционал приложения включает полноценную систему создания и редактирования 2D-плана здания с поддержкой инструментов для рисования стен, создания комнат, размещения мебели, ламп и датчиков. Разработана автоматическая система генерации 3D-моделей стен и пола на основе 2D-контура, а также механизм процедурной генерации текстур (кирпичная кладка, дерево, дверь) без использования внешних графических файлов. Для обеспечения точности построений реализована система привязки к точкам (snapping) и проверка коллизий при размещении предметов.

Важной особенностью системы является реализация трёх режимов симуляции работы устройств умного дома. Случайный симулятор демонстрирует работу датчиков в случайном режиме, полезен для проверки визуализации. Симулятор «Умный дом» реализует логику автоматизации: включение света при обнаружении движения в тёмное время суток, управление кондиционером на основе показаний температуры, аварийный режим при задымлении. Энергосберегающий режим автоматически выключает свет при длительном отсутствии движения и достаточном есте-

ственном освещении. Ручной режим позволяет пользователю через кнопки на панели управления вызывать события движения и дыма, а также включать, выключать и переключать лампы.

В приложении реализована комплексная система хранения данных на основе SQLite. Разработана нормализованная структура базы данных, включающая 13 таблиц для проектов, стен, комнат, предметов мебели, устройств, истории состояний, расписания автоматизации и сценариев освещения. Для обеспечения целостности данных реализованы внешние ключи с каскадными действиями, а также индексы для оптимизации производительности запросов. Предусмотрена возможность сохранения проектов как в виде JSON-документа для быстрой сериализации, так и в нормализованном виде для детального анализа.

Разработанное приложение представляет собой эффективный инструмент для проектирования систем умного дома, который способен удовлетворить потребности как профессиональных проектировщиков, так и обычных пользователей. Система обеспечивает наглядное представление планировки помещений, визуальную обратную связь от устройств (изменение цвета ламп, отображение показаний датчиков), а также удобный интерфейс для управления автоматизацией.

Перспективы развития проекта включают внедрение импорта реальных 3D-моделей в формате OBJ/FBX для повышения реалистичности сцены, реализацию полноценного динамического освещения с тенями (Shadow Mapping), добавление поддержки виртуальной реальности (VR) для иммерсивного просмотра, разработку мобильного приложения для удалённого мониторинга состояния умного дома, а также интеграцию с реальными IoT-устройствами через протоколы MQTT или REST API.

					<i>МИВУ.09.03.02-06.000 ПЗ</i>	<i>Лист</i>
<i>Изм.</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Подп.</i>	<i>Дата</i>		45

Список использованных источников

- 1 Шилдт, Г. Java 8. Полное руководство / Г. Шилдт. – 9-е изд. – М.: Вильямс, 2015. – 1376 с.
- 2 Хорстманн, К. С. Java. Библиотека профессионала. Том 2. Средства продвинутой разработки / К. С. Хорстманн. – 10-е изд. – М. : Вильямс, 2018. – 992 с.
- 3 Stemkoski, L. Developing Graphics Frameworks with Java and OpenGL / L. Stemkoski, J. Cona. – 1st ed. – Boca Raton : CRC Press, 2022. – 298 p.
- 4 Блинов, И. Н. Java. Методы программирования / И. Н. Блинов, В. С. Романчик. – Минск: Четыре четверти, 2013. – 896 с.
- 5 Davison, A. Killer Game Programming in Java / A. Davison. – 1st ed. – Sebastopol : O'Reilly Media, 2005. – 969 p. – Режим доступа: <https://vdoc.pub/documents/killer-game-programming-in-java-5kh5av4msve0> (дата обращения: 10.03.2026).
- 6 Кузнецов, С. Д. Базы данных / С. Д. Кузнецов. – 2-е изд. – М. : Академия, 2018. – 496 с.
- 7 LWJGL – Lightweight Java Game Library [Электронный ресурс]. — Режим доступа: <https://www.lwjgl.org/> (дата обращения: 10.03.2026).

Приложение А

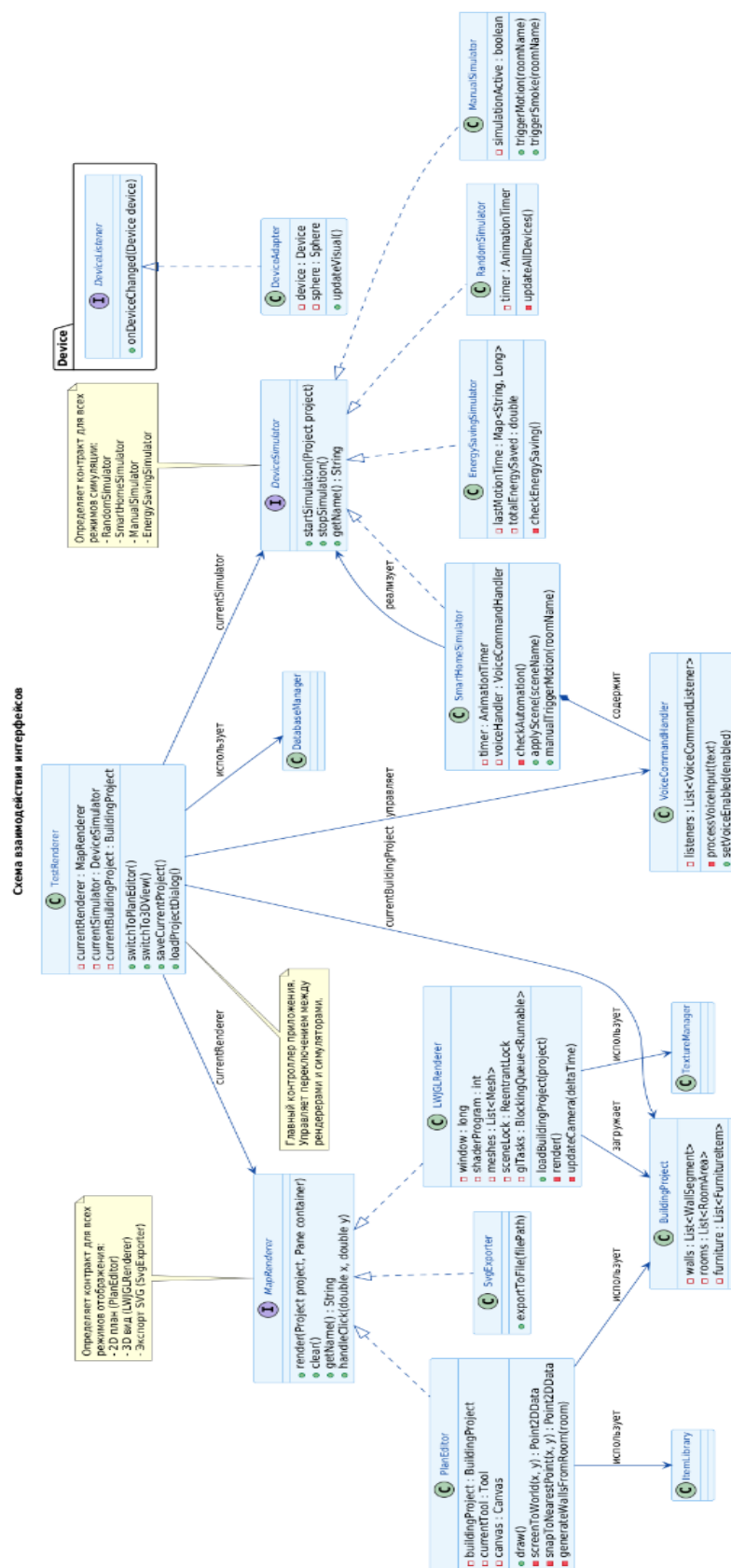


Рисунок 8 – Схема взаимодействия интерфейсов

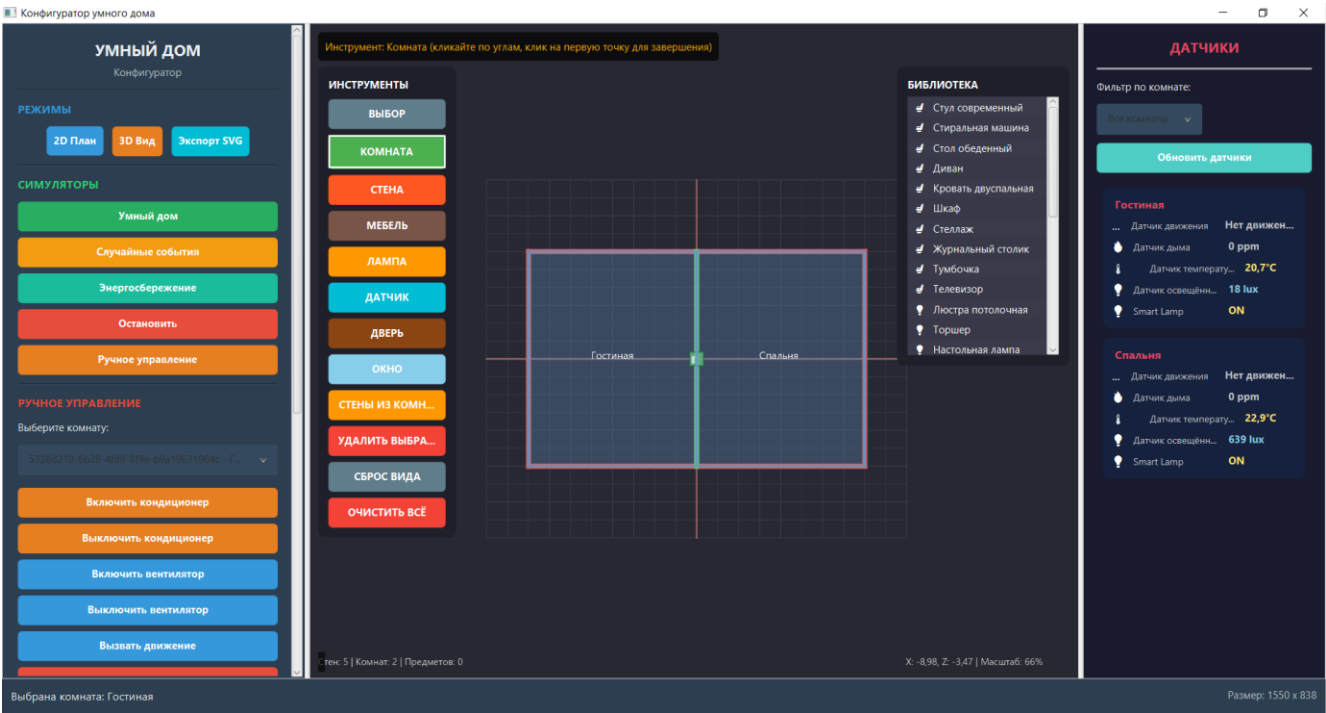


Рисунок 9 – Вывод результата теста №1

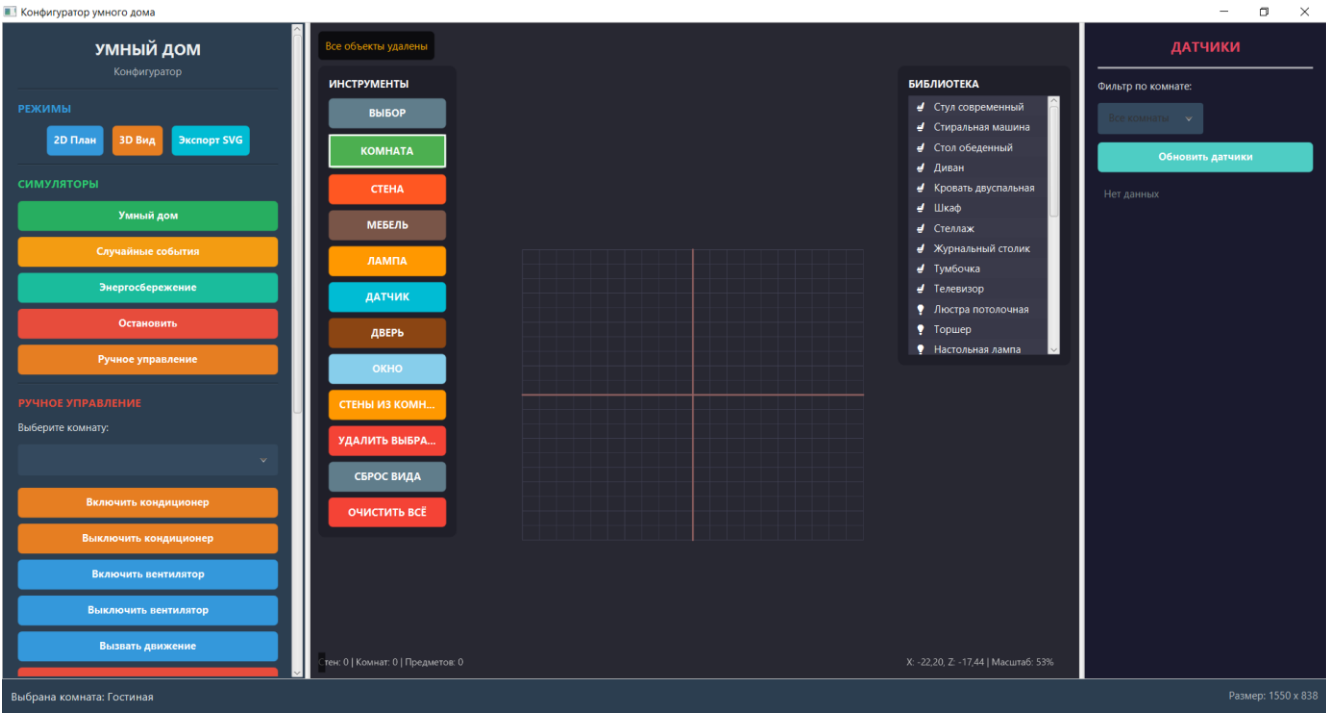


Рисунок 10 – Вывод результата теста №2.

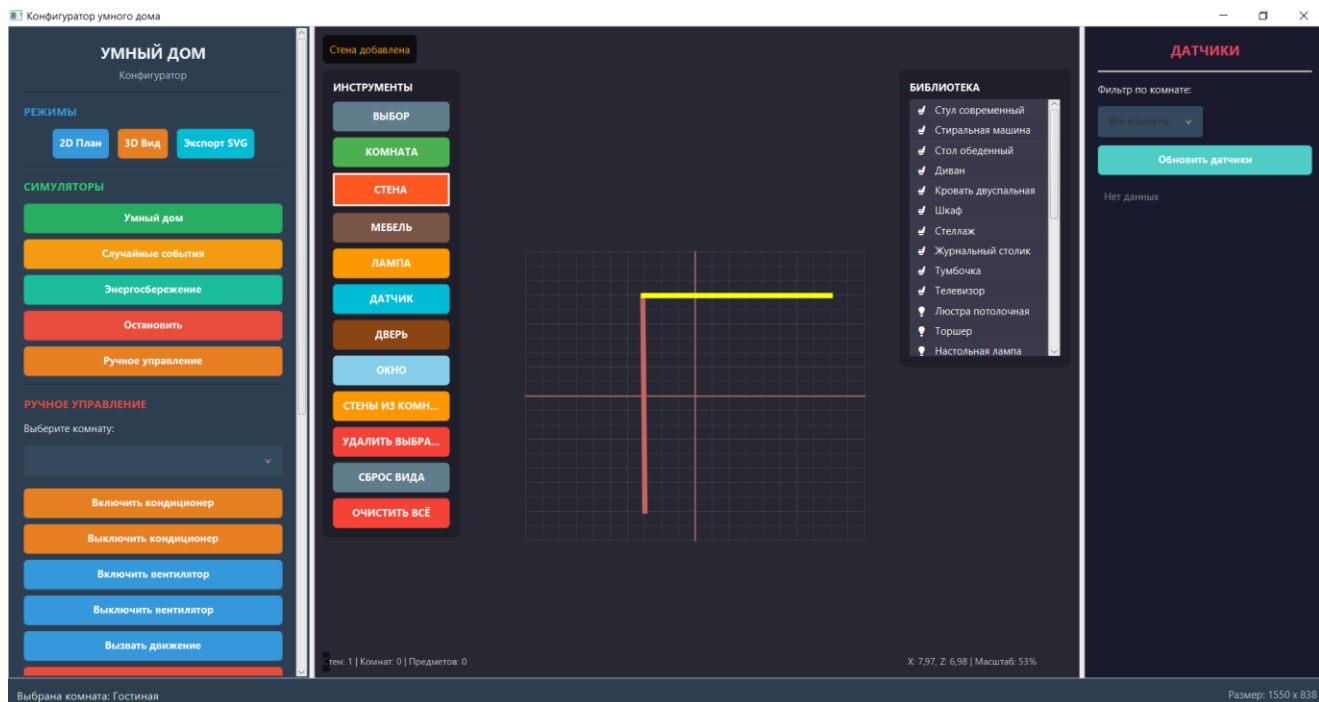


Рисунок 11 – Вывод результата теста №3

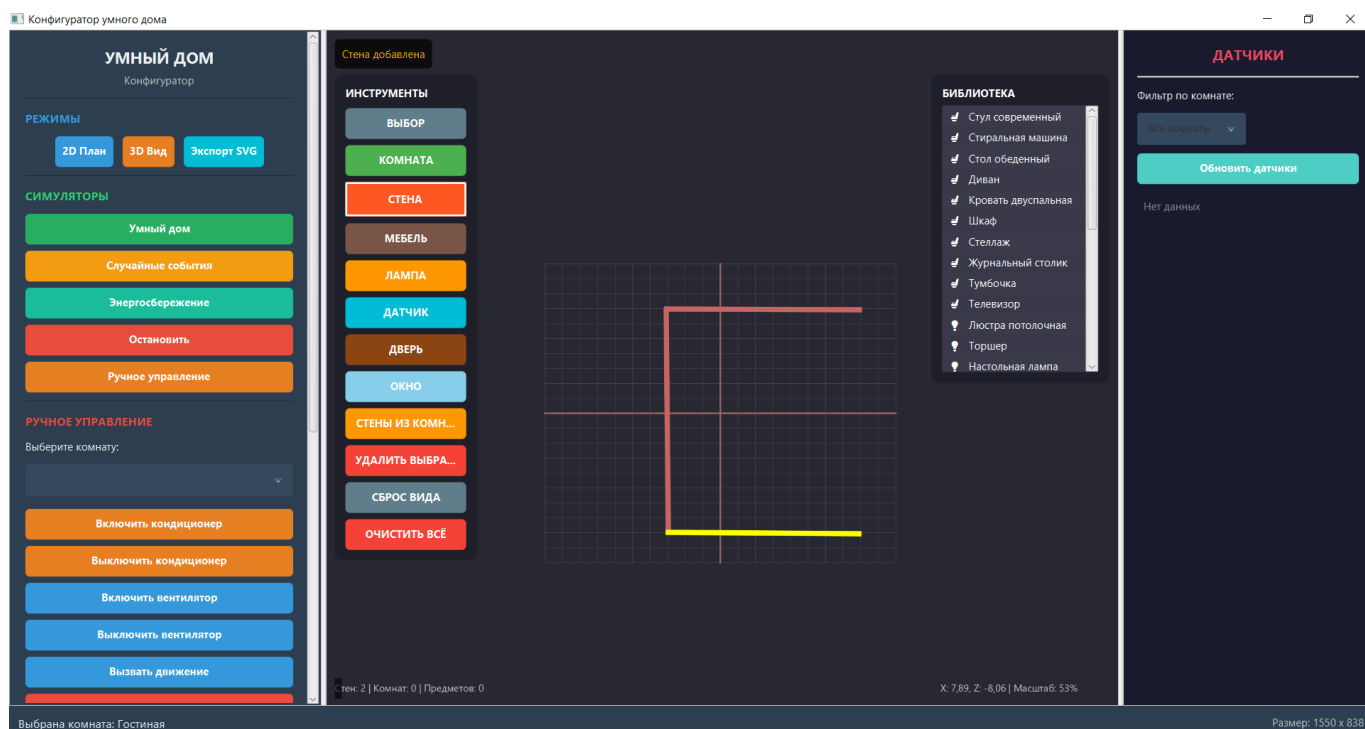


Рисунок 12 – Вывод результата теста №4

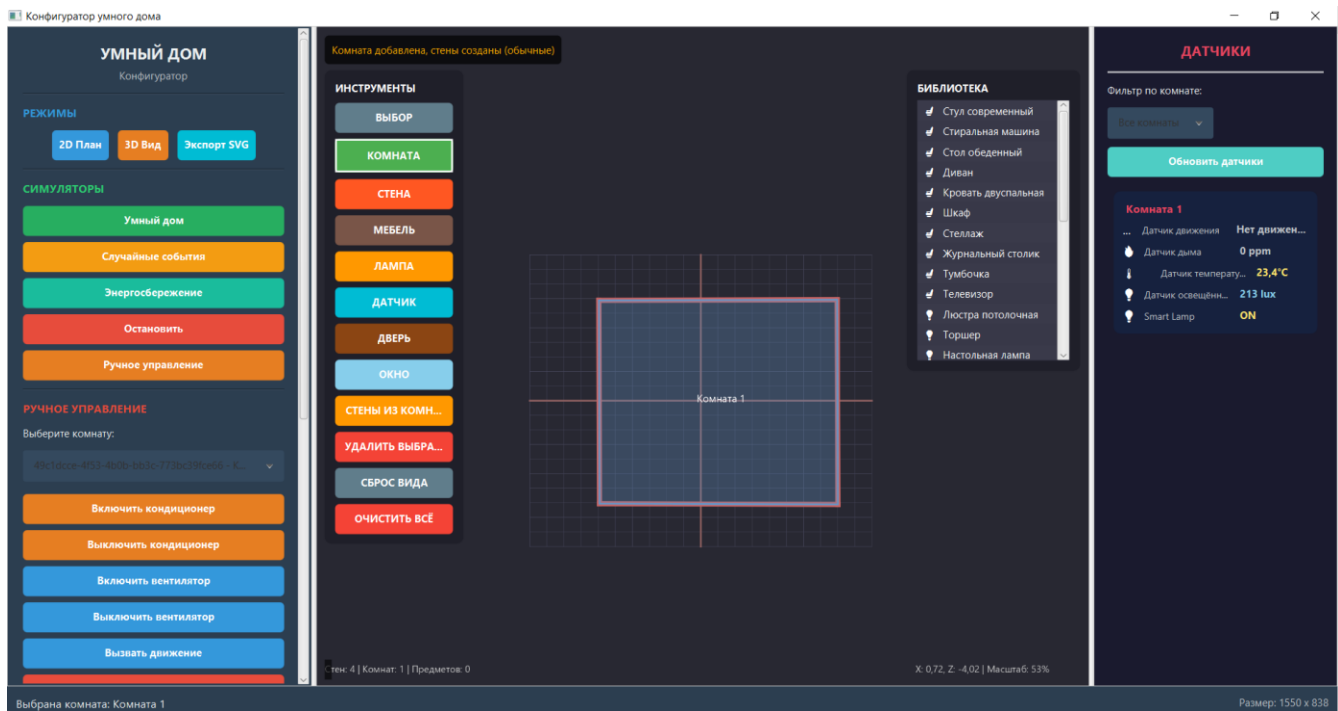


Рисунок 13 – Вывод результата теста №5

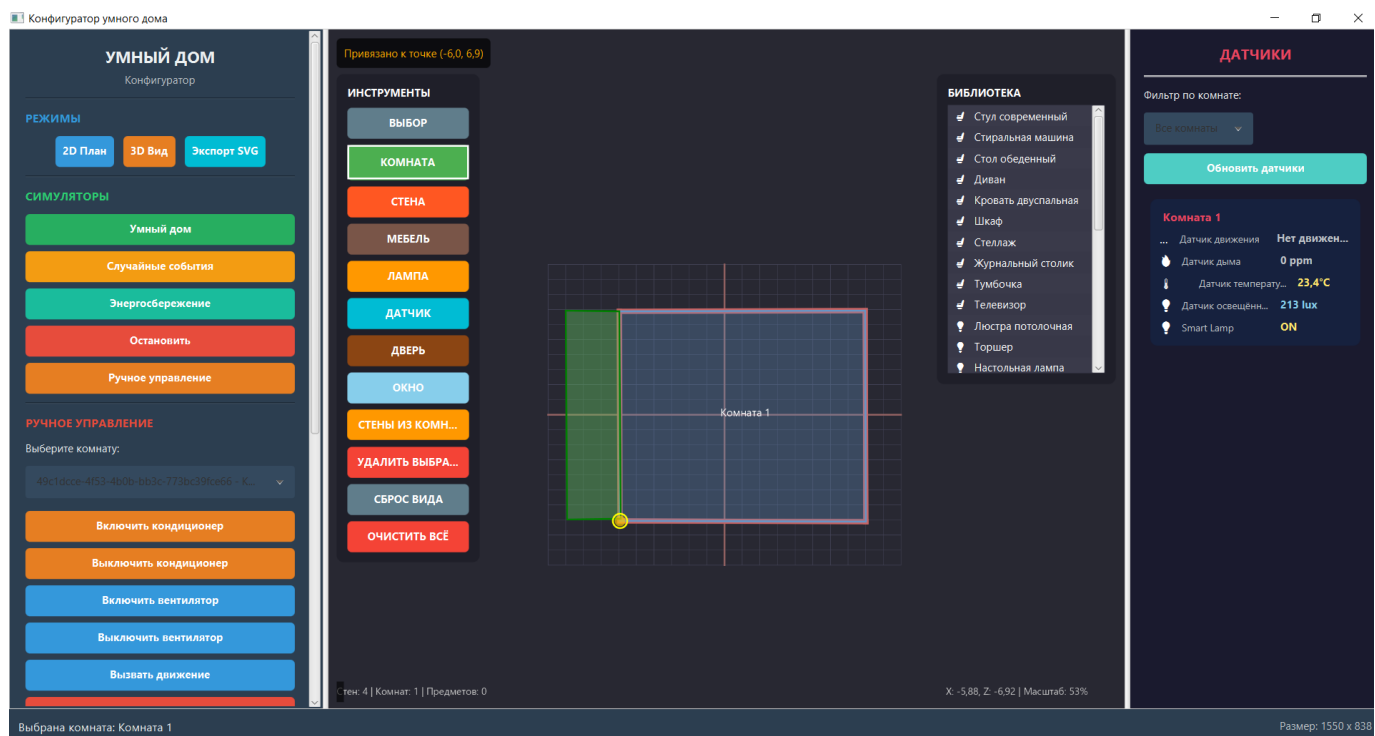


Рисунок 14 – Вывод результата теста №6

					МИВУ.09.03.02-06.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		50

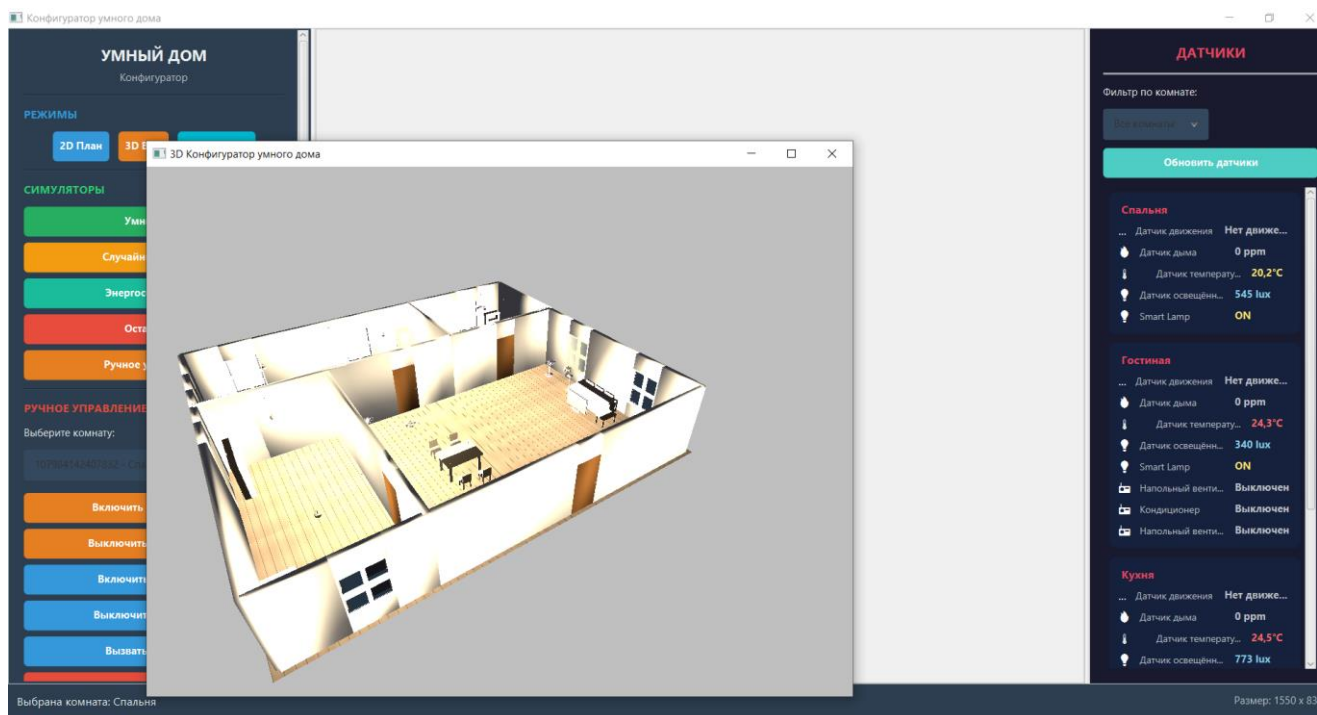


Рисунок 15 – Вывод результата теста №7

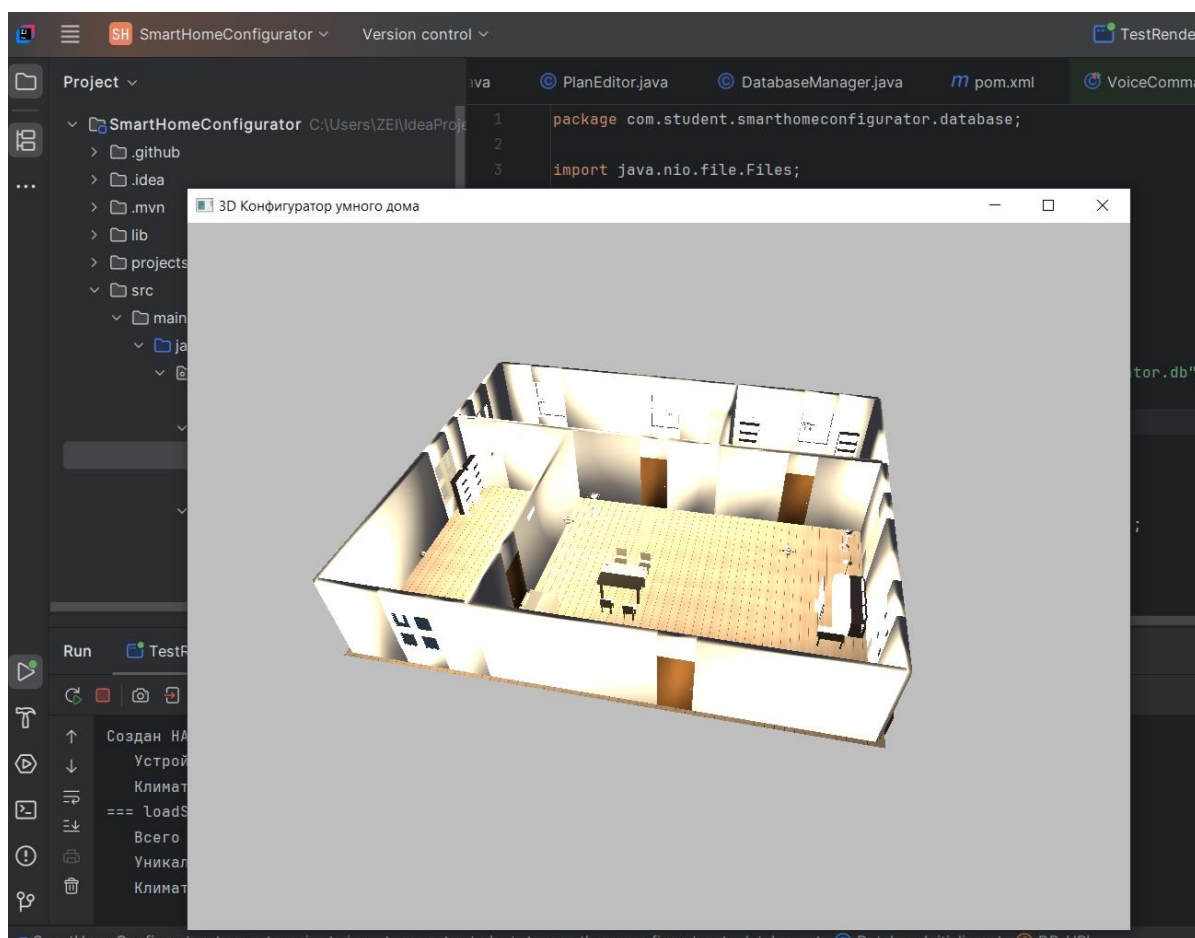


Рисунок 16 – Вывод результата теста №8



Рисунок 17 – Вывод результата теста №9

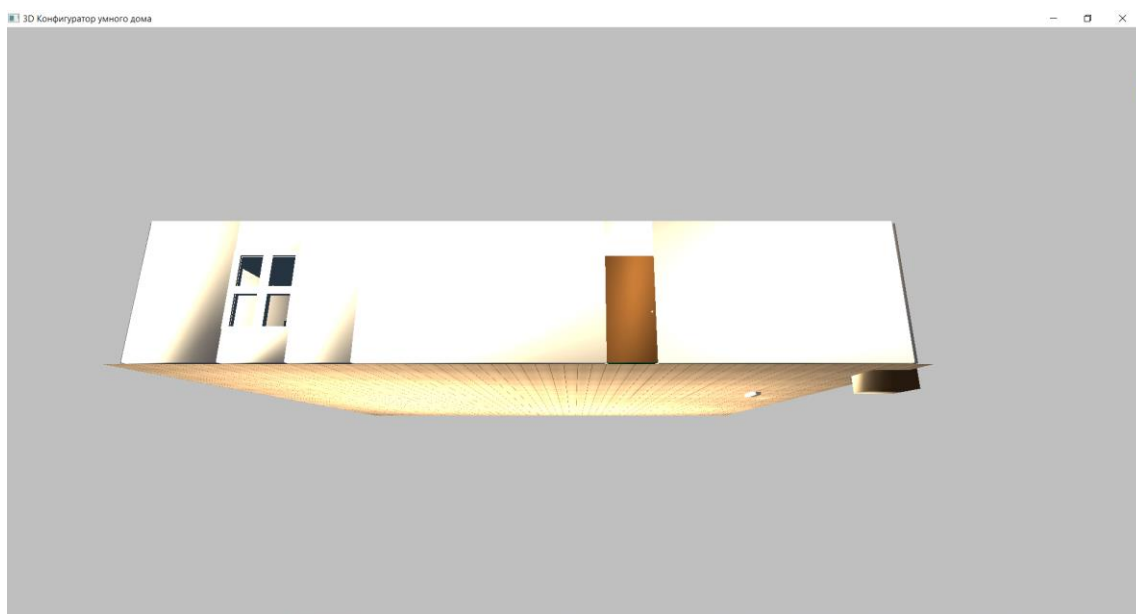
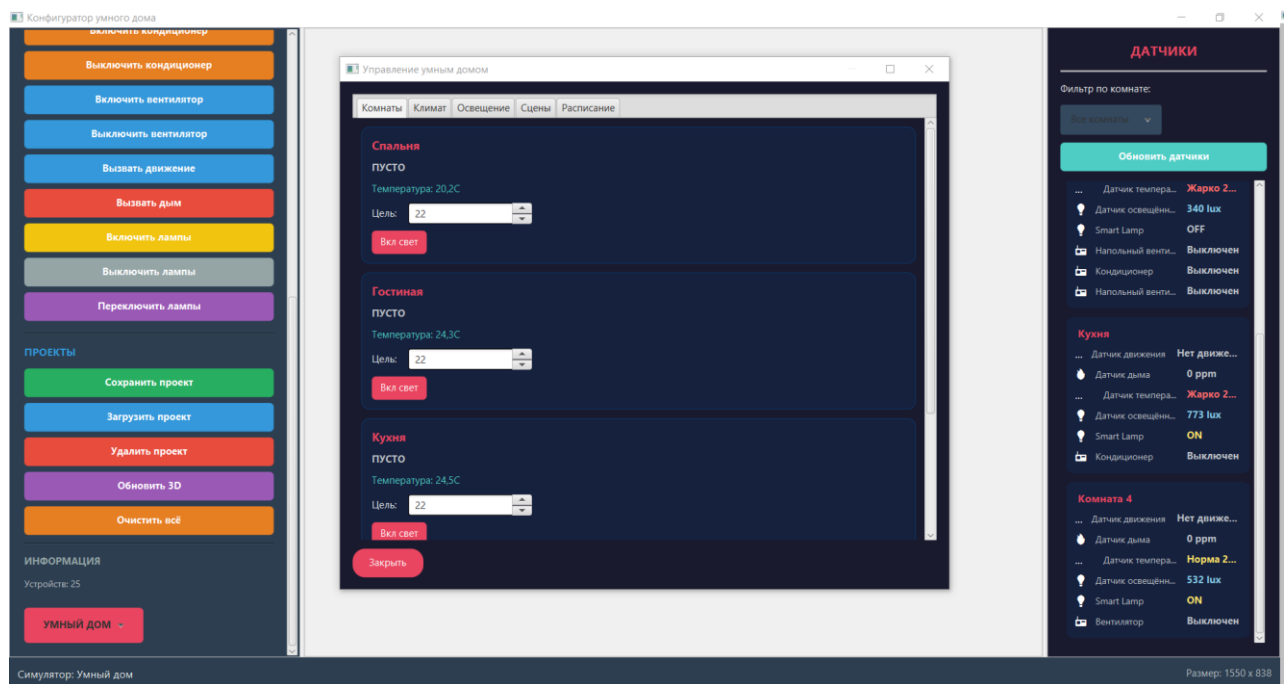
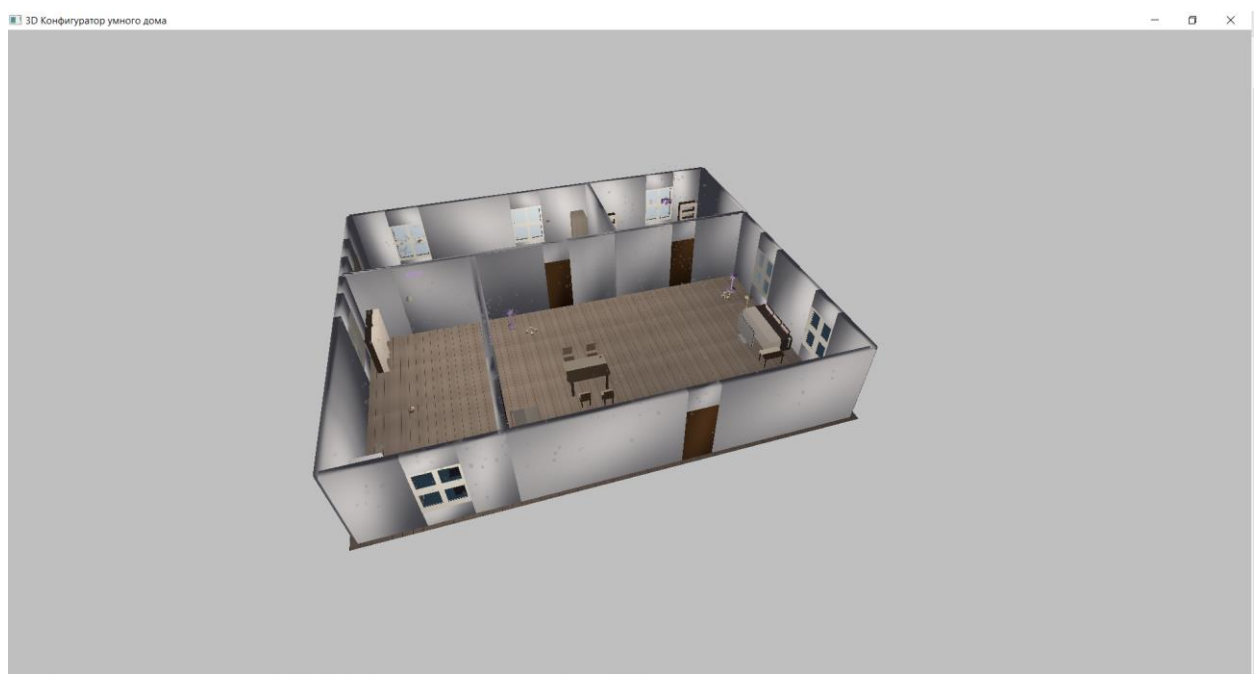


Рисунок 18 – Вывод результата теста №10

					МИВУ.09.03.02-06.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		52

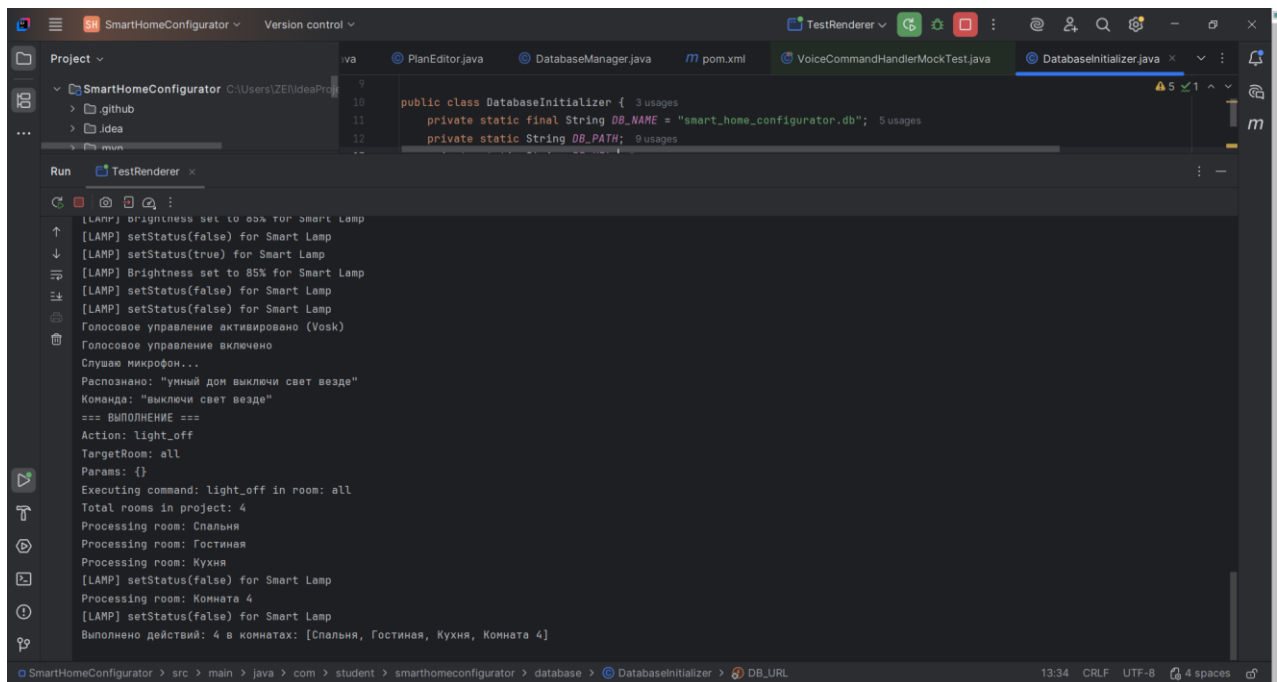


а)



б)

Рисунок 19 – Вывод результата теста №11. Вид с меню настроек (а), вид с 3D (б)



а)



б)

Рисунок 20 – Вывод результата теста №12. Вид с консоли (а), вид с 3D (б)

					МИВУ.09.03.02-06.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		54

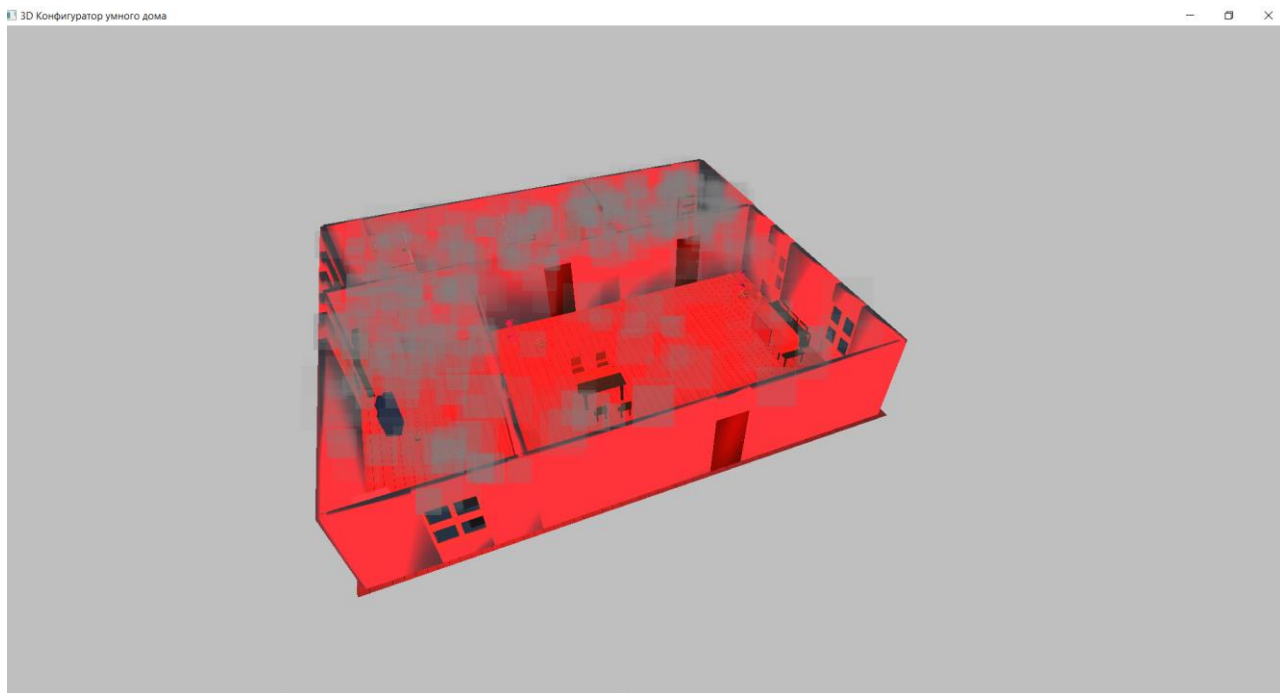


Рисунок 21 – Вывод результата теста №13

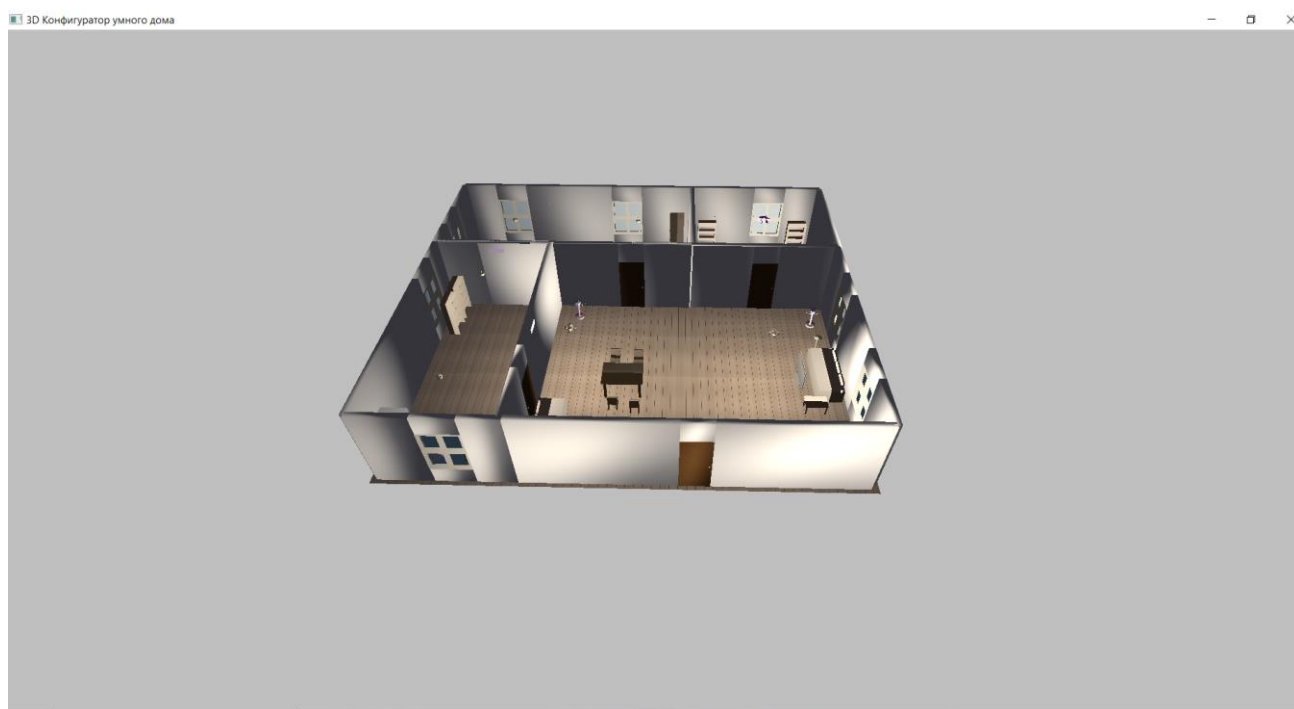


Рисунок 22 – Вывод результата теста №14

					МИВУ.09.03.02-06.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		55



Рисунок 23 – Вывод результата теста №15

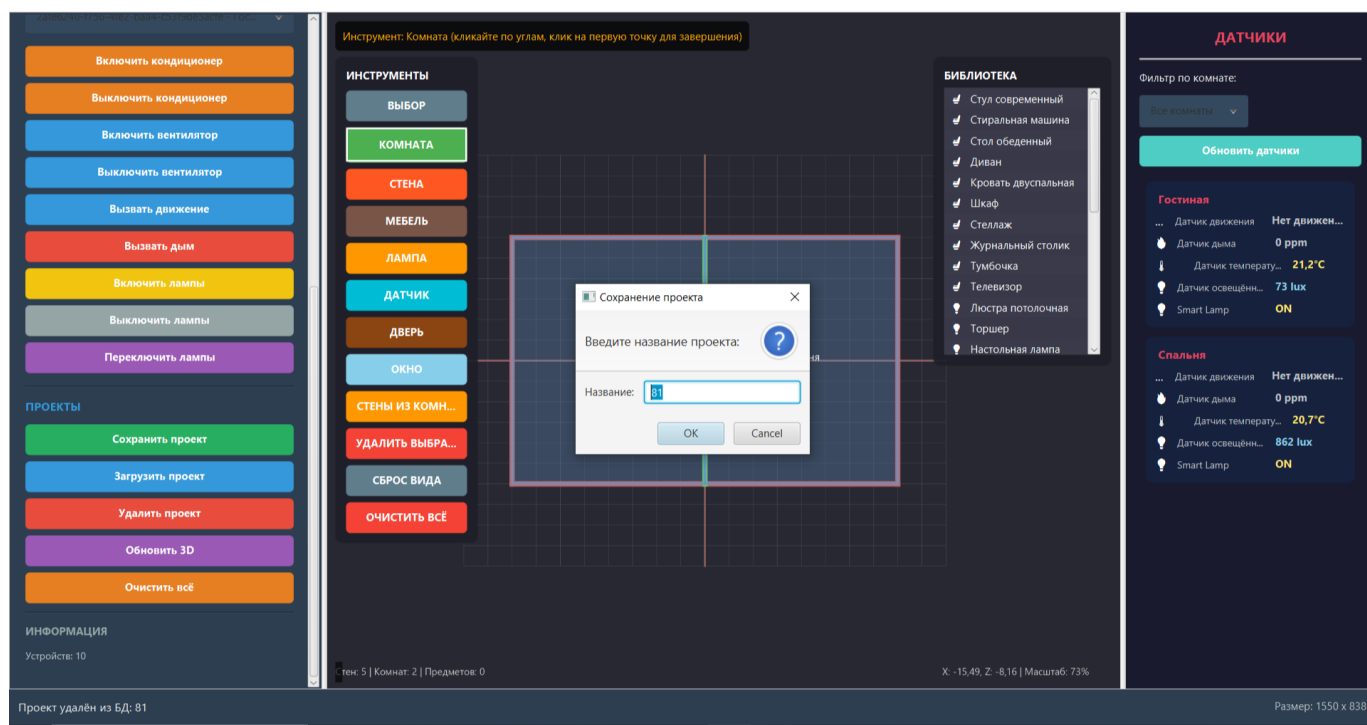


Рисунок 24 – Вывод результата теста №16

					МИВУ.09.03.02-06.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		56

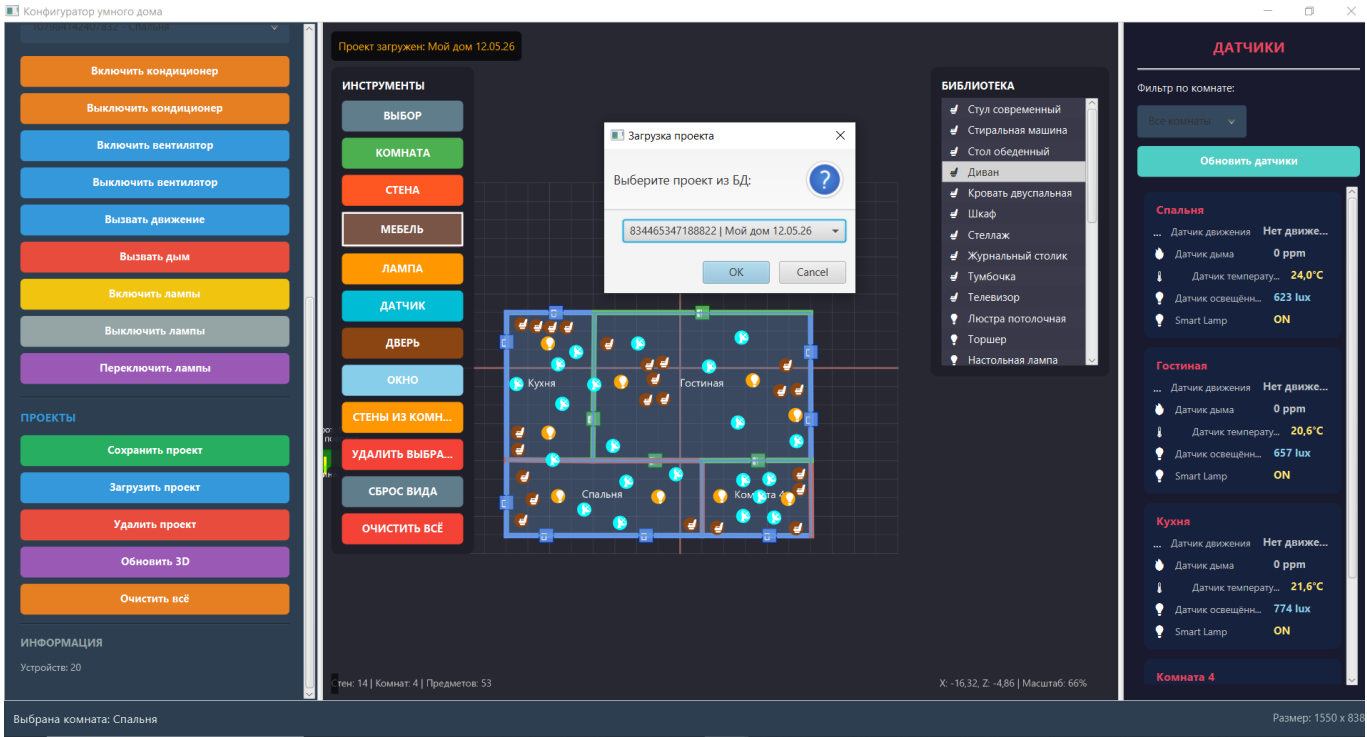


Рисунок 25 – Вывод результата теста №17

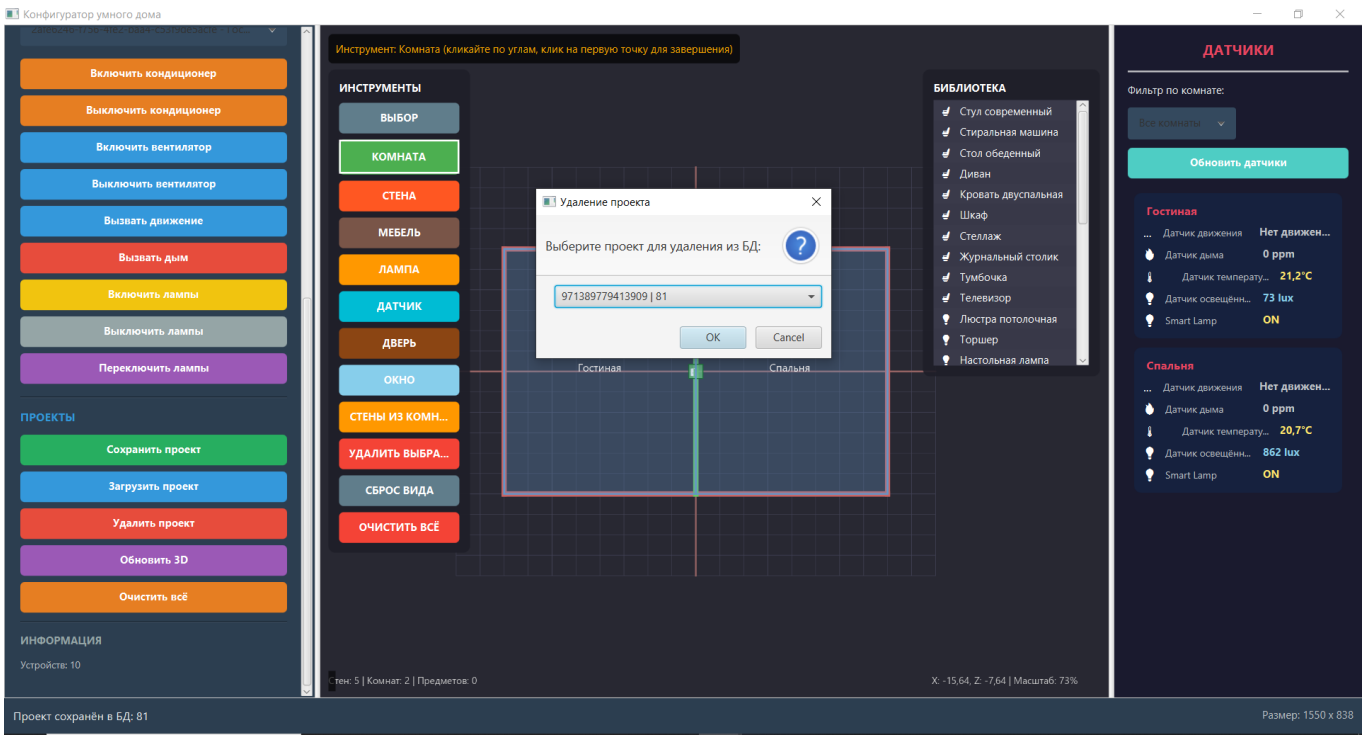


Рисунок 26 – Вывод результата теста №18

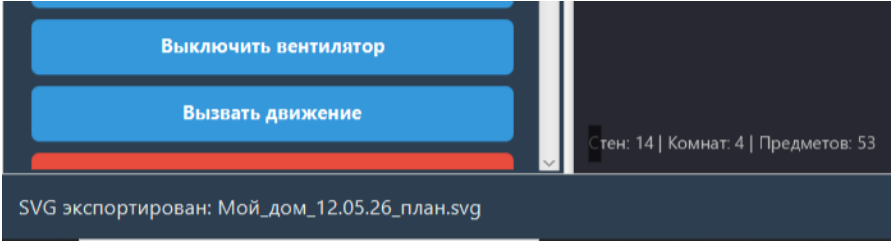


Рисунок 27 – Вывод результата теста №19

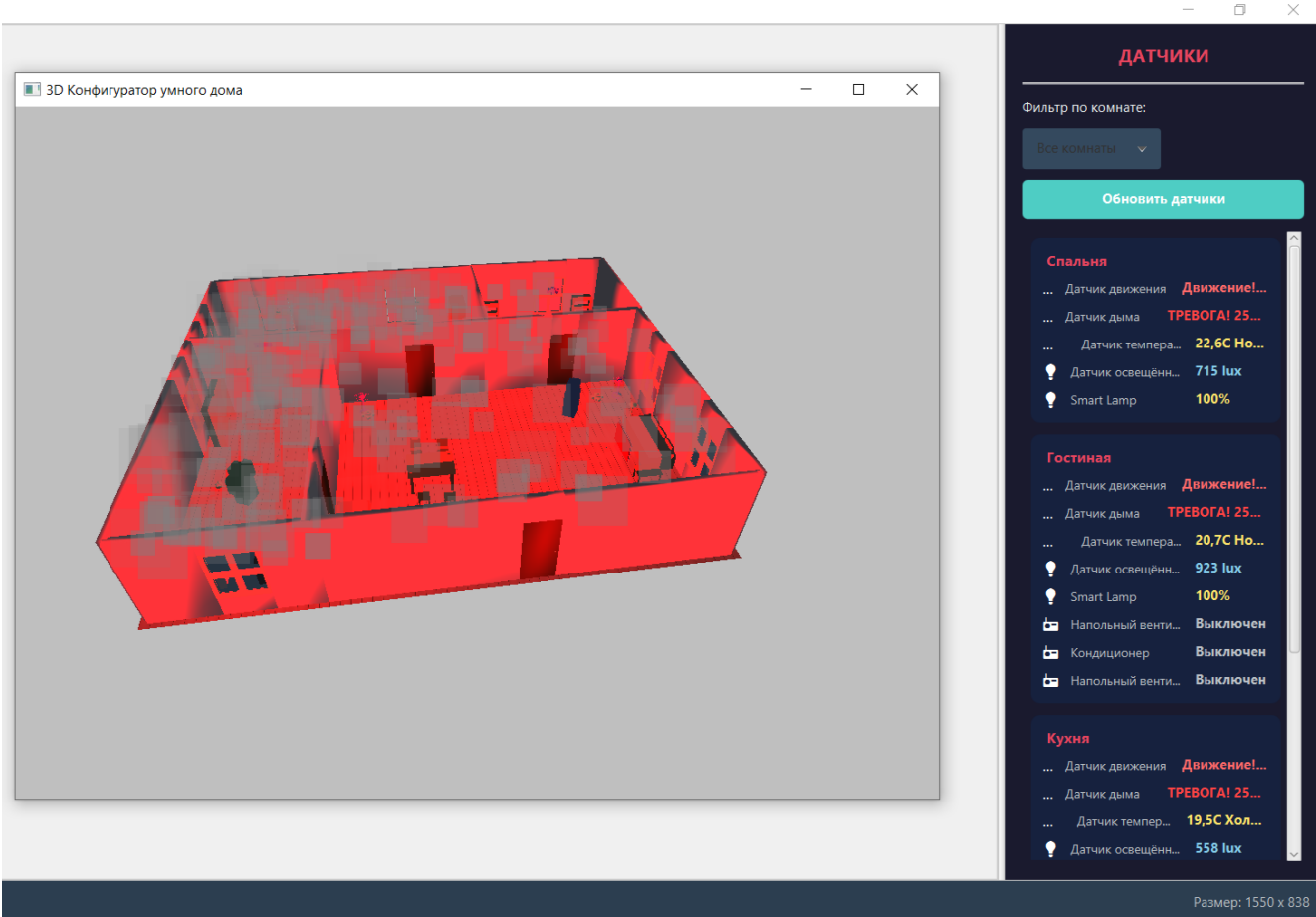


Рисунок 28 – Вывод результата теста №20